ARTICLE | Article in Press

A benchmark dataset for powershell fileless malware detection: methodology, validation, and n-Gram feature extraction

Manish Kumar Meher · Adyasha Rath · Prabodh Kumar Sahoo · Aswini Kumar Samantaray · Ganapati Panda

Received: 25 May 2026 / Accepted: 1 October 2026
© The Author(s) 2026

Abstract

PowerShell is now an integral part of the Windows operating system and includes robust system management, automation, and configuration capabilities. Yet the PowerShell-based malware detection is a challenge as its malicious functionalities can be covered with obfuscation and get executed without any conventional executable artifacts. Many existing detection approaches are limits with proper justification on validation and labeling. Also many of them focus on the detection of PowerShell based malicious script without give emphasis on the preprocessing and feature extraction process. This study proposed a benchmark-oriented detection framework considering the advanced multi-phase deobfuscation, script normalization, VirusTotal based label validation, n-gram feature extraction and Functional Link Artificial Neural Network based classification. The proposed framework is measured using multiple n-gram configurations and standard classification metrics with a regulated train-test protocol. The Trigonometric Functional Link Artificial Neural Network (TFLANN) model achieve an accuracy of 98.61% and 0.9930 ROC-AUC using 5-gram feature sets under the reported 70:30 evaluation setting. Repeated seed trails and paired statistical tests validated the model's rank among the four functional expansions and conventional linear and ensemble classifiers were used to put the reported accuracy into perspective. The result represent that the efficient preprocessing and lexical representation support accurate detection of malicious PowerShell script to a grate extent.

Keywords PowerShell script · Fileless malware · Deobfuscation · N-gram feature · Neural network

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.

Adyasha Rath, Prabodh Kumar Sahoo, Aswini Kumar Samantaray and Ganapati Panda: These authors contributed equally to this work.



https://doi.org/10.1038/s41598-026-74828-2

Published online: 08 October 2026

Introduction

Cyberattacks that have been increasingly advanced through the use of scripting software like Microsoft PowerShell have become a major challenge to the contemporary security architecture[1]. As a valid administrative and automation platform that is part of the Windows platform, PowerShell provides extensive access to the operating system, which attackers frequently use to introduce, install, and manage malicious code[2]. The portability and the malleable character of PowerShell allow assailants to develop very obfuscated codes that can circumvent the standard static detection devices and endpoint defenses. Therefore, creating efficient mechanisms for analyzing and identifying malicious uses of PowerShell has become a pivotal research topic in the sphere of cybersecurity and malware analysis[3].

The cyberattacks on organizations through malware were observed to reach 78%, as per a report published in the year 2021[4]. Recent reports show that among all, the fileless attacks cover 82% of the modern attacks[5]. The malware first appeared in the 1970 s and was initially a simple but malicious type of software that could access sensitive data. It has undergone many changes over the years, with state support and financial assistance from influential institutions[6]. One of the biggest challenges when it comes to the detection of PowerShell-based attacks is the vast amount of obfuscation and encoding that it makes use of. Attackers frequently implement several levels of transformation: Base64, hexadecimal, character replacement, and compression to hide their malicious activities from the analyzers that are not dynamic and from the human eye[7]. These scripts are able to dynamically decode, decompress, or rebuild their payloads at runtime, thus making reverse engineering and feature extraction more difficult[8]. Consequently, productive and effective deobfuscation is a precondition to any data-driven analysis, such as machine-learning-based malware classification and behavior profiling.

Along with obfuscation, the reliability of datasets also remains a major challenge in the area of supervised detection of PowerShell scripts. The existing publicly available datasets provide a large number of samples, which can improve the model training but cannot guarantee reliable labeling, diversity in samples, or duplicate samples[5]. As a result, the inconsistent labeling and validation can affect the training samples and consequently hamper the reliability of the detection model[9]. Therefore, a reliable, systematically constructed, and validated dataset is required for an effective classifier for a PowerShell malware detection model.

In the area of malware detection, ML and DL techniques have shown encouraging outcomes[10]. The existing literature has investigated combining different ML classifiers, convolutional neural networks, autoencoders with feature representations like AST-based, semantic embeddings, behavioral features, and n-gram representations. These approaches show that the lexical and structural features can provide meaningful information to differentiate between benign and malicious scripts. The n-gram feature representation can extract meaningful lexical patterns that represent the link to malicious scripts and their structure[11]. However, the results vary with different studies because of the effect of dataset, deobfuscation, feature representations, train-test splits, and evaluation protocols[12].

Many limitations were identified in the existing studies, like primarily emphasizing developing classifiers while placing less focus on the reliable labeling and validation of used dataset. The deobfuscation is mostly based on AST reconstruction or memory-based analysis, which may not be efficient for multilayer obfuscated scripts. Although many feature representations were implemented, still the systematic evaluation of different n-gram orders remains limited. Also, a lack of benchmark datasets leads to different datasets, and experimental protocols make it difficult to compare existing studies[13,14]. These limitations motivate proposing a framework that addresses dataset preparation, deobfuscation, label validation, feature extraction, and classification by connecting them with each other instead of independent processes.

In order to overcome these limitations, this research proposes a comprehensive system for reliable detection of PowerShell malware. The framework includes multi-stage deobfuscation and script normalization, label validation with VirusTotal, and systematic evaluation of features from 3-gram up to 5-gram, which is accomplished with TFLANN. The proposed approach is compared to baseline models based on a common experimental protocol to

determine its effectiveness and reliability. The study also investigates the feature-order effect, the sensitivity of the parameters, the significance of the results, and the computational cost, and makes a thorough analysis of the proposed detection framework. The experimental evaluation mentions the broader assessment of the proposed framework along with the accuracy and confusion metric. The model performances are evaluated using various classification metrics, including accuracy, precision, recall, F1-score, and ROC-AUC. The effect of n-gram order is also considered in the analysis along with parameter sensitivity, statistical significance, and computational cost. This experimental evaluation clarifies that the observed performance is applicable to the proposed detection framework and whether it provides an alternative to PowerShell malware detection. The primary contributions of this work are as follows:

1. Reliable Sample Labeling: This investigation provides a process to validate the PowerShell scripts using VirusTotal, which defines whether the scripts are admissible or not to train a model that detects real-world malicious PowerShell scripts.
2. Multi-layer Deobfuscation: This paper presents a multi-feature, multi-layer deobfuscation process to retrieve meaning and clean script from highly obfuscated PowerShell scripts.
3. Feature Extraction: To train ML-based models, a meaningful feature set is one of the key components in the process. This study tries to extract meaningful features from the deobfuscated script with multiple feature extraction methods to identify the best technique among them.
4. Proposed ML model: Proposed a novel machine learning implementation, the FLANN architecture of machine learning-based malicious pattern recognition and malware detection, using functional expansion to increase the ability of the model in the description of complex interactions between features in the data.

The remaining portion of this research work is classified into the following sections: Section 2 analyzes various existing approaches and presents various research gaps in them, respectively. Section 3 explains the proposed approach in detail. Section 4 explains the process of n-gram Feature Extraction. Section 5 and 6 presents analyses of various simulation-based experiments, and concludes the work, respectively.

Related work

PowerShell-based malware detection has been studied using various distinct data sources, meaningful feature representations and machine learning and deep learning techniques. Existing literature's has used network traffic, script based, abstract syntax trees, lexical n-grams, semantic embeddings, and behavioral features. However, the dataset quality, de-obfuscation process, labeling of malware samples, and the ability to extract the meaningful features to get the script behavior decides the effectiveness of the proposed approaches. This section reviews representative studies based on these perspectives and identifies the limitations that motivate the methodology proposed in this study.

Literature search strategy and threats to validity

A supervised literature search was conducted to collect pre-existing studies appropriate to PowerShell malware detection using keywords like "PowerShell malware detection," "fileless malware detection," "Malware Dataset creation," "Script Deobfuscation," "Feature representation in malware," "Machine learning in malware detection," "Machine learning in cybersecurity." This review was conducted to find out the methodological trends, common datasets, and limitations in the existing literature. The literature search was carried out mostly using Google Scholar and IEEE Explorer. To search relevant literature, the keywords related to "PowerShell based malware detection" were used in different combinations, like "PowerShell AND malware detection," "PowerShell AND fileless malware," "PowerShell AND dataset," "PowerShell AND deobfuscation," "malware detection AND machine learning," and "PowerShell AND n-gram." Backward and forward snowballing were also used to

identify additional relevant studies. Grey-literature searching is also used to identify different theses, technical reports, government/industry reports, etc., relevant for this work. The objective is to minimize the threats to validity by choosing meaningful and related keywords to extract useful literature. As a result, the review of related work will cover most relevant areas where PowerShell-based malware detection limit and try to cover those limitations through the proposed workflow.

PowerShell and fileless malware detection

In early and recent eras, other than the conventional executable files, fileless malware can be identified as a major threat and growing research area. Demmese et al.[15] collected monstrous network-session data, approximately 3.6 million sessions, 5,321 of which were malicious, of all kinds, such as Wildfire, APT samples, Palo Alto customers, and Cobalt-strike profiles. The authors transformed the raw payloads into grayscale images and used CNNs to detect fileless malware and Cobalt Strike beacons and achieved a rather good accuracy. According to the dataset-creation perspective, the big wins are attracting a variety of real-life sources and the ingenious notion of converting binaries into images to allow spatial learning to take effect. Nevertheless, it will make trouble labeling fileless data; as it has to squeeze variable-length traffic into fixed-size images, and some of the original sequence data is lost when it is squeezed into an image. Generally, this piece of work makes us remember that it requires huge, well-marked corpora and clever representations that keep the signal alive to the machine learning models. Again, the syntactic and semantic characteristics can not be extracted directly from the network-level representations. So these limitations direct the importance of script-level representation of malicious PowerShell scripts, which preserves the characteristic representation.

Bozkir et al.[16] collected memory dumps, converted them into images (even in red-green-blue format), and detected fileless malware with that, yielding the fileless dataset, Dumpware10, of 4294 samples of many different families. The authors examined binary-to-image and introduced manifold tricks such as Uniform Manifold Approximation and Projection (UMAP) for feature analysis. The lesson learned in terms of data sets is that memory artifacts are a treasure trove of spot-fileless detection and that image encodings may be worth looking at. On the negative side, it is not a very large dataset, and the fact that it uses snapshots in memory would not have been very useful in other settings. Then, to be strongly detected,it have to combine content representations that are memory representations, network representations, and script representations. It limits the script-level representations where malicious PowerShell script detection is the objective.

Singh et al.[17] replaced a MITRE-attack dataset of 15,764 malicious behaviors and 1,624 benign ones with explainable features and early detection with LSTM variants. To dataset makers, the key items to capture are both annotated behavioral labels and timestamped, sequential features, as this will allow models to not only identify attacks in their early stages but also provide a human with a reason why. The feature explainer helps to get the complete idea of the important features. It performs with an accuracy of 96.84% at the operational stage with their proposed Argus.

Wu et al.[18] used a small data set of 82 benign and 218 malicious files from GitHub and Malware Bazaar and on hooking AMSI to detect pre-execution detection. These findings are encouraging, though the small sample size makes generalization a small matter. The main thing is that, by hooking into AMSI, it can pick up pre-execution artifacts, but it would still be in need of a substantial number of more diverse samples to construct a strong dataset. This work limits with diverse sources and small-size scripts, which may increase the true negative rate for unseen and new malware variants.

PowerShell dataset construction and label reliability

In the field of malware detection and classification, a dataset plays a major role, which requires a quality and diverse dataset for robustness. The quality of a dataset depends on the factors like sample diversity, duplication, preprocessing,labeling accuracy, and class balance[19]. Alahmadi et al.[20] built a PowerShell corpus with

approximately 12,000 total samples from PowerShell Gallery and GitHub and used a stacked denoising auto-encoder to detect maliciousness. Their findings were good: high true positive and low false positive. However, it also demonstrated that the dependence of detection performance lies on the size and variability of the data. Moral of the story: it should have a balanced set with a high number of obfuscation styles and harmless samples to ensure it does not overfit. Also, the labeling procedure of the dataset must be transparent and details about the procedure must be mentioned.

Tsai et al.[21] gathered more than 3000 samples and integrated AST de-obfuscation with multi-label profiling. The systematic command profiling and fine-grained label granularity, which contribute to making the data more useful to multi-label classification, are the contribution here. The authors are advising preparing datasets as follows: Ensure the samples are labeled and have metadata such as command types and semantic tags to allow the supervised models to capture more specific patterns. But for labeling of data, the authors do not mention any authentic procedure to label them as malicious or benign samples.

Mimura et al.[22] collected data from Hybrid Analysis, AnyRun, and GitHub and used the token/character word-embedding techniques to vaporize static PowerShell. Their writing demonstrates that careful tokenization and embedding can put variable length and a dash of obfuscation under control; however, it still require larger malicious sets and a strict false-positive test when creating reliable datasets.

Fang et al.[23] combined manual feature engineering with FastText semantics, scraping public repos and Cobalt-Strike scripts. The accuracy is good, but it is observed that their approach relies heavily on de-obfuscation. The point here is that syntactic and semantic feature sets that are hybrid require clean, de-obfuscated input, and this brings us back to the topic that de-obfuscation is the key to reliable dataset features. The feature extraction depended on the de-obfuscation effectiveness.

Kim et al.[24] studies have focused on deriving meaningful and discriminative characteristics of the obfuscated JavaScript and PowerShell scripts in order to increase the identification of malicious code. Researchers were able to compile a complete range of thirty-two different features, based on both behavioral and structural features, which enabled the creation of a solid dataset to be used in the further training and testing of the models. Many machine learning and deep learning models have been tested, but one of the models, Light Gradient Boosting Machine (LGBM), was able to perform better in the dataset all the time. External representations, including AST-based, n-grams, word-level n-grams, byte-level n-grams, and fuzzy hashing, were added to add more feature diversity, including syntactic and semantic patterns. Continuous validation is also noted in the literature as a requirement, and therefore, the future of this research will be focused on the inclusion of diverse malicious scripts that have never been observed before; this will enhance the generalization of the model and guarantee its reliability in the real world of detection.

The above studies collectively state that the size of a dataset alone is not sufficient to prepare a reliable benchmark dataset. Diversity in source, malware family, preprocessing, obfuscation techniques, and, most importantly, a proper validation process must be included. In particular, the absence of proper sample validation procedures in some existing datasets drives the need for an independent label validation phase in the proposed framework.

Deobfuscation and structural analysis of PowerShell scripts

In case of PowerShell-based malware obfuscation, a major challenge is implied with different encoding techniques. The malicious scripts can be concealed using encoding, string manipulation, compression, aliasing, character substitution, and nested transformations.

Xiong et al.[25] prepare an AST-based de-obfuscation pipeline and semantic signature scheme based on a benign set of approximately 2,342 benign samples and a malicious set of approximately 4,141 malicious samples. The paper addresses the data quality step directly: with the help of replicating the AST and extracting semantic bits, the obfuscated scripts are converted to the refined script. The moral is to overlay structural (AST) and repetitive steps of decoding the script whenever the dataset is going to be prepared.

Tsai et al.[21] also used AST-based deobfuscation with command-level profiling. This process shows that it is more effective to recover script structure before the feature extraction can increase the informative representation of multi-label classification. But in the case of heavily transformed scripts, effective recovery of the underlying structure depends on the ability of the AST-oriented parser and deobfuscation procedure.

Zhang et al.[26] propose a powerful malicious PowerShell script detection algorithm that closely combines the deobfuscation with syntax-driven feature extraction. The method involves deobfuscation, which extracts hidden or obfuscated behavior from memory dumps of PowerShell scripts before analysis. Once a clean script has been obtained, a special subtree-splitting algorithm is applied to the Abstract Syntax Tree (AST) to produce fine-grained structural and semantic command features. These subtree fragments obtain local node semantics and global sequence relationships. These features are then coded into the network by a two-layer neural network to capture delicate malicious patterns. The effectiveness of multi-source data with experiments on a self-constructed multi-source dataset indicates accuracy 98.87% and high robustness to adversarial samples, which support the utility of complete deobfuscation in conjunction with AST-based feature extraction.

Yang et al.[27] studies of malicious XLM-based document analysis present the importance of structured deobfuscation and multi-view representation learning as urgent. The empirical investigations demonstrate that XLM malware will often use layered obfuscation and thus requires multi-stage deobfuscation pipelines that simulate the execution flow of Excel in order to fully reveal hidden logic. AST-based parsing and formula-type-driven handler selection techniques have been shown to be useful in reconstructing execution sequences.

The literature indicates that in PowerShell malware detection, deobfuscation must be considered an important integral component instead of an optional preprocessing phase. However, the existing literature emphasizes deobfusction mechanisms or structural representation of the scripts. To improve effective feature extraction, a broader multi-stage strategy can be proposed which capable of applying diverse decoding and transformation operations to provide more effective output.

Feature representation for malicious script detection

In the field of ML, feature representation plays a major role and influences classification performance. In PowerShell malware classification, it is importance to extract meaningful features from the script for efficient classification. Therefore, different literature investigated different feature representation processes to get better results.

Song et al.[11] proposed an artificial intelligence-based technology with a feature engineering technique to enhance the performance of malicious script detection. The researchers analyzed the PowerShell scripts with a statistical approach and preprocessed them with a token and abstract syntax tree (AST)-based method for the feature selection phase. For feature extraction, different n-gram method combinations were used and evaluated using machine learning (ML)-based models. Among all the models, the ML model with the 3-gram of the selected five tokens and the DL model with experiments based on the AST 3-gram show the best performance with an accuracy of 98%. For future work, the deobfuscation of obfuscated PowerShell scripts and optimized feature extraction techniques may be used along with effective robustness. This literature is limited as lexical patterns extracted before the deobfuscation process may not be efficient to represent the underlying malicious behavior.

Yang et al.[27] also has focused on the significance of rich feature extraction, which is able to extract various behavior patterns. Word-level representations provide textual clues, whereas the token-level features represent functional intent through formula keywords. AST-based features do not lose structural semantics, and relational graph features represent behavioral and functional relationships. The cross-view combination of these diverse feature sets is always able to improve the detection rate, and as such, it is easy to detect intricate XLM malware. This could also be implemented in PowerShell script detection, deobfuscation, and feature extraction for improved machine learning model-based malware detection.

Fang et al.[23] demonstrated the benefits of using the combination of manually engineered features with semantic embeddings. The literature suggests that the lexical and semantic representations can efficiently capture the required information. However, the effectiveness of the feature quality mostly depends on the preprocessing and

deobfuscation pipeline, which must be capable of recovering the encoded hidden script's content from the obfuscated script before proceeding to the feature extraction.

Kim et al.[24] emphasize the importance and usefulness of multiple feature representations, which include AST features and different n-gram techniques along with fuzzy hashing techniques. Such representation captures different aspects of malicious behaviors from the scripts, which combines the local patterns to structural and approximate similarity relationships. Nevertheless, an increase in the number of features leads to an increase in dimensionality and computational requirements. Therefore, systematic evaluation of feature representation provides an appropriate balance between the detection performance and computational cost.

Machine learning and deep learning for malware detection

In malicious script detection, both ML and DL approaches have shown considerable potential. ML models such as gradient-boosting algorithms, can perform effectively with engineered lexical and structural features, while DL models can learn the nonlinear patterns directly from sequential or structured data.

Alahmadi et al.[20] demonstrated the effectiveness of an autoencoder with stacked denoising features for PowerShell malware detection. It indicates the effectiveness of representation of learning when the size of the data set is large and diverse. Singh et al.[17] demonstrated the usefulness of LSTM-based models for sequential behavioral detection of PowerShell malware, especially when the scripts contain the temporal information. Zhang et al.[26] represented that the neural architectures can be more useful while exploiting structural information extracted from deobfuscated AST representation. These studies jointly represent that the DL can learn complex relationships that may be difficult to identify in manually engineered features alone.

In the meantime, the ML-based models remain popular in the case of static script classification, as they can provide efficient training and inference if discriminative lexical features are available. Kim et al.[24] reported an admissible performance with the LightGBM model using engineered features. Song et al.[11] also demonstrated the usefulness of conventional ML models with an n-gram feature set, which can achieve high classification performance.

The broad area of cybersecurity literature also emphasizes the increasing importance of AI and ML for detecting evolving cyber attacks[28]. The reviews of malware detection techniques highlight the role of adaptive representations and robust detection mechanisms for changing attack behavior[2]. In the research area on AI-enabled cybersecurity, there is more emphasis is placed on the evolving complexity and automation of cyberattacks[29]. The broader reviews in IoT Security similarly emphasize that heterogeneous data, evolving attack areas, and limited generalization remain persistent challenges for intelligent security systems[7,30]. The above observations are relevant to PowerShell malware detection because malicious scripts can also evolve through multiple obfuscation techniques, behavioral variation, and new attack techniques.

The current evidence indicates that neither ML nor DL is always superior in all PowerShell detection scenarios. On the contrary, representation quality makes a difference with performance, a dataset, preprocessing, and evaluation protocol. This inspires systematic comparison of lightweight ML comparisons of appropriate baseline models through a common dataset and testing process.

Multi-stage obfuscation and multi-view malware analysis

The evidence of multi-stage obfuscation and multi-view malware analysis is also found in the literature's beyond the PowerShell malware, which substantially complicates the malware analysis. Yang et al.[27] investigated XML-based malicious documents and intensified the significance of structured deobfuscation and multi-view representation learning. The literature demonstrates that to acquire hidden logic from the layered obfuscation script, the multiple processing stages required, which act in a synchronous mode. This study further stated that the word-level, token-level, AST-based, and relational feature representation can give additional information for efficient malware analysis

Despite the fact the target area in[27] is XML-based malware instead of PowerShell, but still the underlying methodological observation is applicable, i.e., a single decoding might not be sufficient to extract the malicious logic concealed through the multiple obfuscation layers. This motivates the consideration of a multi-phase preprocessing scheme for PowerShell scripts, where different decoding, transformation, normalization, and reconstruction techniques are applied to extract concealed malicious logic before feature extraction.

The broader cybercrime and cybersecurity studies also highlighted that the modern attacks regularly combine multiple techniques to evade the conventional detection mechanisms[31]. Therefore, the malware detection frameworks must consider both the reliability of the input representation and the ability of the detection model to generalize beyond previously observed attack patterns.

Comparative analysis and research gap

The above reviewed literature shows considerable progress in the area of PowerShell and Fileless malware detection, however, several areas still require improvements.

First, much literature's primary focus is on the detection models while giving very limited details on the reliability of the used dataset[20,21]. Although the large datasets are valuable, dataset size alone does not demonstrate or emphasize the accurate labels, sufficient diversity, or absence of repeated and incorrectly categorized samples.

Second, although many studies emphasize deobfuscation, still the approaches often emphasize a particular representation, such as AST reconstruction or memory-based analysis[27]. Multiple obfuscations techniques may be involved in highly obfuscated PowerShell scripts, including encoding, compression, string manipulation, character substitution, aliasing, and nested payloads. Therefore a multi-stage deobfuscation strategy must be required before the feature extraction.

Third, existing literature's implemented diverse feature representations, which include ASTs, embeddings, behavioral features, and images along with n-grams[11,24]. Although these representations have demonstrated great accuracy and performance individually, systematic evaluation of different n-gram orders using a consistently preprocessed and independently validated PowerShell corpus remains important.

Fourth, the reported performance values are frequently based on different datasets, train-test partitions, evaluation protocols, and feature sets. Thus, a direct comparison of the reported accuracy values is not possible. The results of separate studies can be misleading[11,24,26]. Controlled experiments involving the same data set, preprocessing to fairly assess competing models, pipeline, and an evaluation protocol are required.

Based on the above observations (summarized in Table 1), the main research gap is not the absence of ML or DL techniques for detecting malware, but rather the lack of an integrated and systematically validated workflow that takes into account the reliability of the dataset, multi-stage deobfuscation, label validation, feature representation, and controlled evaluation of classifiers for PowerShell-based fileless malware[14]. To fill this void, the present study proposes a pipeline that consists of multiple stages of deobfuscation and normalization of PowerShell scripts prior to label validation and feature extraction. In order to enhance the reliability of the malware/benign

Table 1 Comparison of Existing Studies with the Proposed Study.

Existing work	Dataset Reliability	Multi-stage Deobfuscation	Label Validation	3–5-gram Comparison	Same-protocol ML Comparison
Demmese et al.[15]	✓	×	Partial	×	×
Xiong et al.[25]	✓	✓	Limited	×	Limited
Alahmadi et al.[20]	✓	Limited	Not central	×	✓
Tsai et al.[21]	✓	✓	✓/Ground truth	×	×
Fang et al.[23]	✓	Dependent on preprocessing	Limited	×	✓
Zhang et al.[26]	✓	✓	Limited	×	✓
Song et al.[11]	✓	Limited	Limited	Partial	✓
Present Study	✓	✓	✓/VirusTotal	✓	✓

labels, VirusTotal-based validation is employed; systematic 3-gram to 5-gram representations are explored to see how lexical granularity affects classification. The resultant representations are tested with TFLANN and relevant baseline models, with a standard experimental procedure. This design allows the reprocessing, representation, and classification stages to be evaluated independently, as the improvement in the performance of the final classifier is not only attributed to the classification stage.

Proposed system workflow

This section addresses the working of the proposed framework in the process of constructing and validating the newly constructed dataset for PowerShell fileless malware detection. It explains the step-wise description of the collection of samples, deobfuscation process to get the clean script, and feature extraction to extract meaningful features to construct a reliable dataset to train and test the ML and DL models.

Overview of the proposed framework

To enhance the malicious PowerShell script detection, a system architecture was proposed consisting of five major parts, including data collection, data cleaning, feature extraction, and dataset creation for PowerShell-based malware detection[6]. The graphical representation of the proposed system architecture was shown in Figure 1.

Dataset collection methodology

Obtaining malicious samples that are authentic and reliable to make a dataset is a challenging task. It's back-breaking to find a good number of admissible samples from public platforms. Those samples carry a significant threat of being misused against systems or individuals. As they possess threats, they must be used with the utmost care and in an isolated environment.

PowerShell sample files

The PowerShell scripts are combinations of specific commands of Windows PowerShell. It was used to perform operations through the PowerShell terminal. Most efficient in the administration-level operation. The scripts are used by typing on the terminal or using a file to execute the command. When the length of the script increases, it is efficient and easy to use a script file. To make PowerShell scripts executable, the scripts are stored in a file format, i.e.,.ps1. Like other scripting languages like HTML and JavaScript, all of them have a file format to store the script and run the code; PowerShell scripts use the.ps1 file format[32].

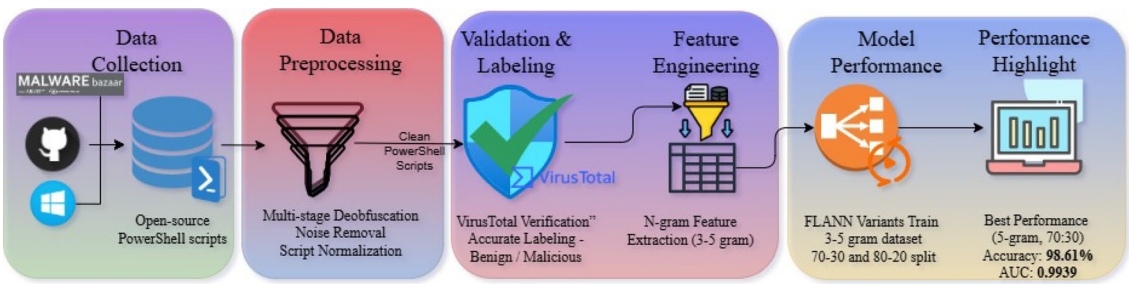


Fig. 1 Proposed system workflow for dataset preparation and validation.

It is very convenient to use this file to run the commands on other computers; it just have to provide the file path in the PowerShell terminal. It was initially designed to benefit the developers to perform tasks like automation, system administration, and system configuration. It has now also been widely used by the attackers to attack and gain access to the system. It may also contain commands that can be used with a malicious intention to hamper or steal information from the targeted system.

The most common use of a.ps1 file in a system is to manage system administration tasks like automation, disk monitoring, and software installation. Security auditing like log creation, vulnerability assessment, and user activity auditing[33]. File folder management, like organizing files and folders and copying and pasting. Task scheduling to run scheduled jobs to automate the processes.

According to Microsoft, the standard file format for the PowerShell script is.ps1, which is commonly used in the case of PowerShell scripts. Like the.ps1 file, the.psm1,.psd1,.ps1xml,.cdxml, and.pssc files are also used for different purposes to perform specific tasks. The.psm1 file is a PowerShell module file that contains functions and classes defined, which are imported into other scripts. The.psd1 file is a PowerShell data file that mostly contains metadata about the defined modules. The.ps1xml file is a PowerShell XML file that defines the format of objects. The.cdxml PowerShell file defines WMI/CMI-related cmdlets. The.pssc PowerShell file is a session configuration file that manages and configures the PowerShell sessions for remote users or systems.

Source of samples

The availability of admissible samples is not common in public repositories. So 5,000 malicious and 5,000 benign samples from sources like Kaggle[34], GitHub[35], and Malware Bazaar[36] were collected. These are reliable and trusted to get the most authentic samples to prepare a robust dataset for PowerShell-based malicious script detection. The Microsoft Authenticate PowerShell script was also collected to increase the dataset quality and performance. There are various predefined scripts available for tasks like system configuration. So Microsoft-verified samples were also collected from GitHub to include in the dataset.

To ensure the quality creation of a multipurpose dataset, obfuscated PowerShell scripts are considered. Obfuscation is a common and effective technique employed by attackers to evade the firewall and antivirus. By keeping real-world challenges in mind, these obfuscated PowerShell scripts were incorporated to create the dataset. For the quality and utility of the dataset, a Python-based deobfuscation technique was employed. It extracts most of the actual script to get better results and analysis at the time of feature extraction. An obfuscated malware sample was displayed in Figure 2.

Multi-stage deobfuscation

Deobfuscation is a process of extracting meaningful and actual scripts hidden inside an obfuscated script. It is an intentional encryption process to avoid the traditional antivirus software. The motive is to extract the meaningful scripts from the obfuscated script for enriched feature extraction. To perform the analysis of obfuscated PowerShell scripts for different attack types, a Python-based deobfuscation tool was designed. This tool transforms the heavily obfuscated PowerShell script into a suitable and interpretable form for analysis and feature extraction. The DE-obfuscation process is carried out to extract the suitable script from the highly obfuscated PowerShell script. The proposed deobfuscator executes through multiple iterations to extract and progressively reconstruct obfuscated PowerShell scripts. Each phase is the combination of normalization, decoding, and content aggregation for better performance and understanding. It also combines additional Indicators of Compromise (IoCs) to identify forensic artifacts in the reconstructed scripts.

```
iex([systeM.TExT.EncoDInG]::utF8.GetstRINg([systEM.COnvERt]::FromBasE64STrIng
('CiR3`Wk9jTHZqe`lVj`TmtUeEF`4b`lNtaTl5Vm`1`Pa1`RYWTZtVU9Xc0ZzTU`pSeGxsTzM0bHJ
`Kam9ENG`cwZ`UppeTR`C`QTd1ZF`l`NdU`pSNXl4WTI1eW5qTGJq`Y1h`MOG`pqZD`NaZldhU0R`T
b243QkRnck9`ySk`Y`1`U`U1MS`29`FYU5KbzZKU0tP`UVcxdl`Ba`Q`3F`GbnRJUk51Z`G`Nj`Wkh
`RcWM3Z`W`l0M09C`cVdxS`z`N1`W`XlBV0t`CSF`B`YTzZG`dn`N6ck`U2T2`o5WnpacE1FMl`F0`
U3`pF`ZW9G`c`ms0SjV`Ma1p3Q`jBy`S`3hV`U2hvZUIgP`SAn`W0RsbElt`cG9`ydCg`ia2Vybm`V
sM`zIuZGxsIildcHVibGlj`IHN0YXRp`YyBleHRl`cm4gS`W50UHRyIF`Zpcn`R`1YWx`BbGxvYyh`
JbnR`Q`dHIgbHBBZG`RyZ`X`NzLCB`1a`W`50`I`G`R3U2`l`6ZS`w`gdWludCBmbEFsbG9jYXRpb2
5UeX`B`lLC`B1aW50IGZsU`HJv`dGVjdCk`7W0RsbEltcG9y`d`Cgia2`VybmVsMzIuZ`GxsIildc`
HVibGljIHN0Y`XRpYyBle`HRl`cm4gS`W50UHRyI`E`Ny`Z`W`F0ZVRocmV`hZCh`JbnRQdHI`gbH`
BUaHJl`YW`RBd`HR`yaWJ`1`dG`VzLCB`1a`W50`IGR`3U3R`hY`2tTa`XplLCBJbnRQdH`IgbHBTd
G`F`ydEFk`ZHJl`c3MsIEludFB0c`iB`s`cF`Bhcm`Ft`ZXRlc`iwg`dW`lu`dCB`kd0NyZWF0aW9u
R`mxhZ3Ms`IE`lud`FB0ciBscFR`ocmVhZElkKTtb`RGx`s`SW1wb3`J0`KCJt`c3Z`jcnQu`Z`Gxs
`I`ildcHV`ib`GljIHN0YXRpYyBleHR`lcm4gSW50UHRyIG1`lbX`NldChJbn`RQ`dHIgZGVzdCwgd
Wlu`d`CBz`c`m`MsIHV`pbnQgY291bnQp`O`yc7JHlra`WJDN`1`JJ`MTV`ReWRlNExtRjI`y`YmFS
`N3`hoZWRhRWVY`dUF0bVRr`U`k`pISTFETmxTd21`Z`N`jJsN3F3R`lB6e`TF6`YmxISWt`1dk9na
`2`Qw`Ul`F`XbEow`Vj`V6MkR`4dVR`Jb0`RKM`kVU`RFNTWnhFYn`VRYVRLaGtmeWJIU1MzS`TlNa
F`NMc`HA`zQ202Wng`3ZEplc`Uh`Y`cU`9EWkdt`cUZId`ldJbW`5iaGh`5Q`kZ`Sa`E9ne`E`h1RU
```

Fig. 2 Raw obfuscated PowerShell script.

Iterative decoding

To counter the multi-level deobfuscation in the obfuscated PowerShell script, the proposed deobfuscator used multiple iterations to extract the refined script. It compares the newly generated script with the output of the previously refined script. It stops the iteration when the scripts of the last two iterations match or the max limit of 10 iterations is hit. This iterative structure was effective for multi-stage PowerShell malware, where Base64, hexadecimal, and compression layers are embedded repeatedly in the script. The multi-stage obfuscation helps the scripts to slip through the basic, single-layer deobfuscator, which bypasses the meaningful features.

Normalization phase

To remove the basic symbolic obfuscation for the obfuscated script, the normalization phase improves further deobfuscation. It helps the advanced deobfuscation function to work efficiently. In every iteration of the process, the input script passes through different phases of normalization. This normalization phase consists of 3 common phases, such as surface cleaning or structural sanitization. It mostly removes null bytes between characters, which disturbs the analysis. It also removes escape quotes like ' and " as well as empty quotes like ‘ ’, “ ”, etc., to refine the script. It helps to retrieve the basic structure of the script, whose effect shows in tokenization or character visibility. The second phase was semantic recovery, i.e., symbol and escape normalization, which aimed to disguise the distorted command. This phase removes the tick (“), which was used as an escape or line continuation command in the PowerShell script. It also removes the caret symbol as well as the string join (+) symbol, which helps to restore the command sequence. As a result, the hidden command names, URLs, or encoded arguments become visible. In the third phase the logical expansion was performed, which replaces the symbolic abbreviated representations of commands with their actual and explicit equivalents, like iex → Invoke-Expression, iwr → Invoke-WebRequest, etc. This phase ensures the full semantic meaning of the script, which is much more readable as a legitimate PowerShell script. It helps in Natural Language Processing (NLP)/feature extraction pipelines that can now interpret it correctly. The unwanted characters with their names and uses are defined in Table 2.

After removing all the unwanted characters and the 1 st phase of deobfuscation, the obfuscated script is still not ready to pass through the feature extraction process.

Table 2 Script normalization operation.

Operation	Characters	Name of character	Use
removeNull	`x00,\\00`	null Bytes, Escaped Nulls, Null	Byte Injection
removeEscapeQuote	`", '`	escaped quotes	Command Injection
removeEmptyQuote	`" ",' '`	empty quotes	variable substitution, and expression evaluation.
removeTick	`"` `	Tick Mark	Apply Obfuscation
removeCaret		Caret mark	Avoid Signature-Based Detection
joinStrings	`"A"+"B"-`	Double quotes with Plus	String concatenation

Decoding layer

To make an obfuscated PowerShell script, the encoder plays a major role. This helps to encode the script to change its meaning completely. So it is important to remove these common encodings by passing the scripts through decoding mechanisms. Followed by the normalization phase, the proposed deobfuscator applies a series of encoding mechanisms to each script. This encoding applies to the script in every iteration of the process until it is terminated. Each of the encoding mechanisms performs a distinct operation with the obfuscated[25]:

(a)Base64 Decoding: The decoding phase aims to extract the command, payloads, and URLs. These are encoded using a Base64 encoder, which helps to avoid signature-based detection mechanisms. Tracing the regular expression pattern of `[A-Za-z0-9+/=]{30,}` in this particular stage and considering a string length of a multiple of four for further decoding. Then the stripASCII() was used in the considered string to extract the actual command or payload from the encoded string[9].

(b)Decompresson: Sometimes the payloads are base64 encoded and compressed, so it first decodes the base64 encoding and then decompresses them using the zlib library under multiple window configurations, like `15, -15,-zlib.MAX_WBITS` , and `16 + zlib.MAX_WBITS`, etc. It successfully decompressed both of the zlib- and gzip-compressed streams, which reveals the embedded binary data or second-stage scripts as hidden as encoded blobs.

(c)Hexadecimal Decoding: Like Base64 encoding, the hexadecimal encoding used an encoding pattern to encode the ASCII characters. So this module locates the sequence of hexadecimal values by using the `(0x[0-9a-fA-F]{2}[,]*){4,}` pattern. Among the set of converted hexadecimal values, identify and convert from hexadecimal notation to raw bytes, and then finally print it to readable ASCII text[37].

(d)XOR Decoding: The XOR decoder works with simple encryption methods that use bit-wise XOR operations to hide text. It finds sub-strings that can be printed and are at least ten characters long ($[-\tilde{}]tt\{10,\}$) and tries to decrypt them with the default key 0x20 (the space character). The decoded sequence is kept only if it has alphabetic strings that are five characters long or longer, which suggests that it is plain text[38].

Indicator of compromise (IoC) extraction

Most of the file-less payloads were carrying URLs, IPs, domain names, etc., to get the actual executable files. This file can be downloaded after the payload runs on the targeted system. It is very important to identify these IoCs in the script to flag them, as they are also encoded. After detailed obfuscation, the process also uses Indicator of Compromise (IoC) extraction to increase script efficiency. It extracts the forensics artifacts from obfuscated characters to locate and categorize that which indicates malware behavior[39]. The IoC forensic artifacts are as follows:

- URLs: The embedded links of Command-to-Control (C2C) communication or any payload retrieval.
- IP Addresses: Detect the pattern of IPV4 addresses present in the PowerShell script with network connection commands.

- Domain Name: Extract the pattern via alphanumeric order followed by.com,.net, etc. It's grouped matches, and only the recognized domain is considered from the grouped matches.
- Registry Keys: It tries to find the expressions matching the `HKLM\ and HKCU\` registry hives. They are the most targeted area for malware attackers.
- System File Paths: Detects absolute file paths of Windows registries and drives for indicators of pre-existing executables or configuration files.

After applying all the obfuscation methods mentioned above, the deobfuscator prepares a refined and cleaned script, which was previewed in Figure 3.

Script validation and labeling

To ensure the authenticity of the collected PowerShell script files, a validation process to filter the actual malicious and benign samples was performed. This process was carried out with the help of the most trusted and popular malware analysis platform, VirusTotal. It analyzes each PowerShell script present in collected.ps1 files with more than 60 virtual machines. Each file, however it is initially considered malicious or benign, is passed through all the VMs available in the VirusTotal platform for a sophisticated and comprehensive analysis. A list of information from the analysis was extracted to prepare a report that helps us to label the considered malicious and benign samples by applying constraints.

Validation work flow

An automated process was deployed using Python code to perform the validation task. It works with the VirusTotal platform to process the files and validate they are malicious or benign. The Python code extracts the.ps1 files from the specified folder one after another and uploads them to[40]. It processed the file one at a time and prepared the detailed report. This can be performed through the VirusTotal free API, which allows analyzing the file at most 4 times per minute and 500 API calls per day. In the prepared code, a constraint has been set accordingly so no API limit is crossed to avoid the failure of the automated process.

```
$6dA5qcyScglQKMJceAvfpCYXWmLKuckH6hDtFIHMwpCYa4l8p5:ProgramFilesPSPath = Microsoft.PowerShell.
Management\Join-Path -Path $enV:ProgramFiles -ChildPath "WindowsPowerShell"
}
elseif (IsCoreCLR) {
    if (IsWindows) {
    $6dA5qcyScglQKMJceAvfpCYXWmLKuckH6hDtFIHMwpCYa4l8p5:ProgramFilesPSPath = Microsoft.PowerShell.
    Management\Join-Path -Path $enV:ProgramFiles -ChildPath 'PowerShell'
    }
else {
    $6dA5qcyScglQKMJceAvfpCYXWmLKuckH6hDtFIHMwpCYa4l8p5:ProgramFilesPSPath = Split-Path -Path
    ([sySTeM.mANAgeMENt.AUTomaTioN.PlaTFORM]::SelectProductNameForDirectory('SHARED_MODULES'))
    -Parent
   }
}

try {
    $6dA5qcyScglQKMJceAvfpCYXWmLKuckH6hDtFIHMwpCYa4l8p5:MyDocumentsFolderPath = [Environment]
    ::GetFolderPath("MyDocuments")
}
catch {
    $6dA5qcyScglQKMJceAvfpCYXWmLKuckH6hDtFIHMwpCYa4l8p5:MyDocumentsFolderPath = $NUlL
}
```

Fig. 3 Clean script after deobfuscation.

VirusTotal-based labeling

In the first step, MD5, SHA-1, and SHA-256 hash values were collected, which confirm the integrity of the.ps1 files. It takes a 3-minute slot, which provides VirusTotal sufficient time to process the file and prepare the detailed report by extracting the result. After completing all the processing of available files, when the process terminated, it prepared a detailed report in an.xlsx file. To prepare the report from the analysis, the following information about each and every sample uploaded to the VirusTotal platform was collected as follows:

- Filename: The original file name to identify the file after confirming its malicious status.
- Detection Score: The number of VMs that flagged the.ps1 file as malicious.
- Detected Machines: The names of all VM machines that flagged the.ps1 file as malicious.
- Cryptographic Identifiers: For duplicate checking, hash values, including MD5, SHA-1, and SHA-256, were considered.
- Threat Type: Based on the observed behavior and signature, the type of malware is “Trojan,” “Backdoor,”“Downloader,”or“Fileless.”

All the above details were collected and presented to verify and validate the PowerShell scripts for labeling. The representation of the report shown in Figure 4 and a graphical representation of some samples with respect to the number of flags detected are shown in Figure 5.

Labeling

The VirusTotal validates the collected PowerShell scripts through multiple commercial antivirus (AV) engines. After collecting and analyzing all the information for each sample in the report, a threshold was set to maintain confidence to consider a.ps1 file malicious.

Benign Script File Report

Filename	SHA256	MD5	SHA1	Flags Detecte	Detected By	Behavior I
Ben_22.ps1	dbe2dbee	0cb41818f	d3c24a798	0	None	N/A
Ben_52.ps1	5094fc5c1	ea520ed0	fde73f45b	0	None	N/A
Ben_53.ps1	339819ab	fe85eac99	5f5589702	0	None	N/A
Ben_56.ps1	1c7eced5f	d04e68a9	e1fdb1491	0	None	N/A
Ben_63.ps1	b731b6e2	8252754b	a70d2af00	0	None	N/A
Ben_67.ps1	82e3468c	487ecfdef	511e85a2c	0	None	N/A
Ben_72.ps1	80cb0545	186e6173	966667f82	0	None	N/A
Ben_76.ps1	486f824a2	e938d8d3	171cce973	0	None	N/A
Ben_86.ps1	90bb7dd2	5de0d5a5	f604a4379	0	None	N/A
Ben_106.ps1	51a06c9f6	c456cb77f	9dbcd5b1c	0	None	N/A

Malicious Script File Report

Filename	SHA256	MD5	SHA1	Flags Dete	Detected By	Behavior Info
Mal2_1113.ps1	40b8e4be	fc41096	cd7789	21	Cynet, CTX, Sangfor	trojan.pantera/pc
Mal2_1114.ps1	3ddad527	9345fef4	234849	11	MicroWorld-eScan,	pantera
Mal2_1116.ps1	b8e427ed	b496a25	bb9bf9	21	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1117.ps1	e4384987	a64146e	ef7f333	14	MicroWorld-eScan,	downloader.boxte
Mal2_1118.ps1	6c411d25	05566c4	3deb98	15	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1119.ps1	390294fef	22673d8	82dcb3	22	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1120.ps1	a2a62256	e1b5722	577d21	22	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1121.ps1	a4955ba0	3f3aa5d	831b08	21	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1122.ps1	41e1395f	17d406d1	22d866	21	MicroWorld-eScan,	trojan.pantera/pc
Mal2_1123.ps1	3317ee28	9582a5c	c1019fe	21	MicroWorld-eScan,	trojan.pantera/pc

Fig. 4 Subset of the report generated by virustotal.

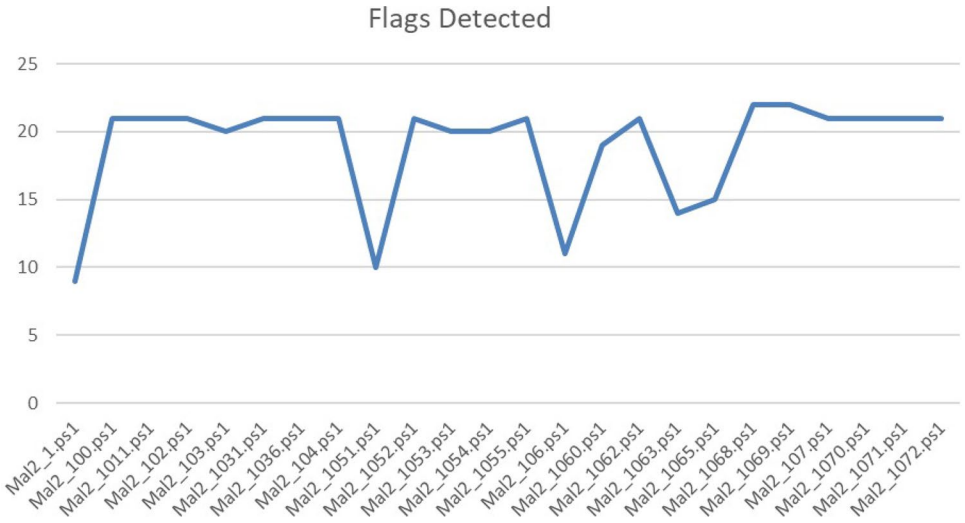


Fig. 5 Number of flags detected with respect to sample.

Consensus threshold

A standardizes the threshold to allow evaluation and separation of the scripts from unacceptable ones. As per the analysis script, VMs flagged by more than 7 are considered malware and collected for further processing. For the benign samples only"zero"-flagged samples are considered. The motivation to take the threshold is to minimize the false positive and increase the quality of the dataset by considering as many authentic malicious samples as possible. The use of a consensus threshold of 7 is only for conservative operational criteria to label the dataset as well as to improve the label consistency. This also maintains the transparency throughout the dataset along with the reproducibility of the dataset construction process. Rather than depending on a single AV vendor, it's better to consider multiple AV vendors for better results.

The rest of the samples that are not considered or that don't match the threshold were discarded to maintain the integrity and reliability of the dataset. To create a robust detection mechanism to detect real-world malicious activity performed by PowerShell scripts, this process not only ensures the training of the model with authentic and well-characterized labeled data but also increases the trustworthiness and accuracy of the dataset, which can be acceptable in further training.

Threats to dataset validation

Although VirusTotal is used extensively for malware validation, it cannot be considered its detection result absolute. As the vendors share information on threat intelligence and employ overlapping detection signatures, which may introduce correlations among AV vendors' detection decisions. The results of vendors may vary over time as they update their signatures and detection models, which results in potential label instability[13].

So to compress the dependence on individual AV engines, this work considers a multi-engine consensus strategy instead of depending on a single AV vendor. Although the considered labeling approach cannot eliminate AV bias or guarantee perfect labels. Still, the threshold of seven can provide a conservative labeling to a transparent and reproducible methodology by minimizing the impact of false positives.

Consequently, the proposed dataset should not be considered as a ground truth labeled dataset; rather, it should be considered as a baseline validation benchmark to support reproducible research[41]. Future work will enhance the reliability of the dataset by means of behavioral sandbox analysis, expert cross-validation, and frequent revalidation.

The current dataset construction workflow emphasizes the collection, validation, de-obfuscation, and preprocessing of PowerShell scripts from publicly available repositories. The current version of the pipeline was not

enhanced with an explicit duplicate or near-duplicate detection stage. Therefore, the same or similar scripts from different sources or repository versions can be found in the collected scripts, despite being stored as separate files. Samples that are common in both the training and testing partitions can cause bias when assessing the model, which affects the model's experimental outcome. As future versions of the dataset are built, content-based duplicate detection using cryptographic hashing (e.g., SHA-256) for exact duplicates and similarity-based approaches (e.g., MinHash or SimHash) was implemented before partitioning the dataset. These improvements will continue to increase the reliability and benchmarking usefulness of the proposed dataset.

n-gram feature extraction

Feature extraction implies the process of extracting selective characteristics known as"features"from the raw samples, which can be used for analysis, detection, or classification by training ML-based models. In cybersecurity, while dealing with suspicious and malicious samples, these extracted features from raw data are helpful to find out pattern presence and association with the normal or malicious files.

In the ML-based classification, the feature extraction from the sample takes a vital role as the model trains using the vector present with the respective feature. In malicious PowerShell script detection, it is more crucial to choose the effective features to be selected to get the best dataset. It leads to getting the best detection accuracy or fewer true negatives. As in the case of malicious PowerShell scripts, it is crucial that no malicious script bypass the security system without triggering a flag.

After the deobfuscation process, it allows us to extract features from the samples more efficiently and accurately, which increases the training quality of the model. As for the feature extraction from the samples, different kinds of feature extraction have been used, covering binary and numeric features. Feature categories like Bypass Features, Downloader Features, Injector Features, Payload Features, Task Execution Features, and Execution/General Features are included for the processing.

To extract the contextual and linguistic structure from the PowerShell scripts, the n-gram feature extraction technique was used. It captures the sequential relationship between tokens along with command syntax patterns and argument orders. Kim et al.[24] and Song et al.[11] defined relationships by capturing the textual and contextual n-gram patterns in the scripts; it counts their occurrence in the script. So in the dataset the number represents the frequency of that particular n-gram pattern in the script. Here, to find out the best feature set, 3-gram to 5-gram feature extraction was performed. By this it could be verified that n-gram feature extraction was efficient among all 5 types of feature extraction using n-gram feature extraction. Here, for each n-gram feature extraction, more than 1000 feature sets were considered to prepare the dataset. The 3-gram to 5-gram captures complex patterns and combinations, which makes it more efficient. All are described in Table 3.

Although the effectiveness of AST-based, semantic, and control-flow features is more admissible, this proposed workflow focused on lexical n-gram feature extraction because of its simplicity, scalability, and robustness. After complication of the deobfuscation process, n-grams effectively extract recurring token sequences and execution patterns that differentiate malicious scripts and benign scripts. Again, n-gram feature extraction needs fewer computational resources and is less dependent on successful parsing than syntax-based representation, making it more effective for large-scale dataset preparation.

Table 3 n-Gram feature categories.

N-Gram	Feature Capture of	Example
3-gram	short behavior chains	invoke-webrequest http malicious
4-gram	extended behavioral context	invoke-webrequest http malicious com
5-gram	complex multi-command behavior	invoke-webrequest malicious com evil start-process.

Table 4 Distribution of malicious and benign PowerShell script samples.

Category	Samples	Size (MB)	Label
Malicious	2260	43.9	1
Benign	2307	5.1	0

Table 5 Summary of extracted dataset and their characteristics.

File name	Feature type	Total features (Columns)	Label column	Filename column	Feature description
`3_gram_features.csv`	Lexical (Trigram)	1001	Yes	Yes	Three-token behavior chains (e.g., `Start-Process FilePath evil`)
`4_gram_features.csv`	Lexical (Four-gram)	1001	Yes	Yes	Extended multi-command context (e.g., `Invoke-WebRequest http malicious com`)
`5_gram_features.csv`	Lexical (Five-gram)	1001	Yes	Yes	Long execution patterns (e.g., full operational sequences)

Results and discussion

This section presents the process and result of the proposed PowerShell malware detection framework. Various simulation-based experiments using different performance metrics are used to evaluate the efficacy of the approach. The experiments and analysis cover the impact of deobfuscation and n-gram features along with the performance of different FLANN models with two types of train-test split.

Dataset description

To prepare the dataset, a total of 4,000 samples have been considered with an equal number of benign and malicious samples. These samples are validated through the VirusTotal platform, where malicious samples are considered only if flagged by 7 or more machines. Then it was cleaned through the deobfuscation process for better feature extraction to train ML/DL-based models. With different sets of features, multiple datasets have been created to find out the optimum feature set to train ML/DL models. The dataset type, size, and label were defined in Table 4.

After finalizing the sample set, the deobfuscation was applied to the script. Then the AST-based syntactic and node-based features and N-gram feature extraction, as well as static and behavioral features, are extracted. Extracted 3 to 5 N-gram feature extractions along with an AST-based and hybrid of behavioral and static features dataset were created. All the details of the prepared dataset are shown in Table 5.

Experimental configuration

To perform the validation and dataset preparation process, all requirements in an isolated virtual machine were set up with Windows 10 Pro, a 64-bit operating system of version 22H2, installed in a system having a processor of Intel(R) Core(TM) i5-6200U CPU @ 2.30 GHz/2.40 GHz and RAM of 8 GB. Python3, along with its various

libraries, was used for the implementation of the approach. To run the Python scripts, Visual Studio Code with all minimal requirements was used.

The experiments were conducted with a relatively low-performance hardware setup to show the approach is not dependent on high-performance computing resources or specialized hardware. The design decision underscores the computational efficiency and applicability of the framework. This allows researchers and practitioners to replicate the results and implement the proposed solution in resource-limited settings with widely available computing resources.

To evaluate the proposed model robustness and reproducibility, multiple random seed values were used to conduct the experiments. These random seeds affect the two components of the pipeline. First, the sample allocation for training and testing results in the different data split for the runs. Second, it controls the randomness of the shuffling during the optimization process, which influences the learned model parameters as well as the gradient updates. So, to provide a more reliable estimation of model performance and effective partitioning of data, multiple random seeds are applied across all models. This influences the results more than the single seed train-test split.

The experimental analysis was carried out using two different train-test partition strategies, namely 80:20 and 70:30. These two configurations were selected to evaluate both learning stability and generalization capability under varying proportions of training data. The dataset consisted of 4,567 PowerShell script samples, including 2,307 benign and 2,260 malicious instances, ensuring a nearly balanced class distribution.

Four FLANN variants were implemented: Trigonometric, Chebyshev, Legendre, and Polynomial FLANN. Each model was evaluated using lexical feature representations derived from 3-gram to 5-gram tokenization. The performance was assessed using accuracy, F1-score, and Area Under the ROC curve (AUC), which collectively measure classification correctness, balance between precision and recall, and class separability, respectively.

Model implementation overview

The proposed classification framework is implemented using a custom FLANNClassifier, which integrates functional expansion with a linear classifier trained using Stochastic Gradient Descent (SGD). The classifier supports four expansion types: trigonometric, Chebyshev, Legendre, and polynomial. In this implementation, the nonlinear mapping is achieved through explicit functional expansion of the input features prior to classification. The expanded feature space is then processed using an SGDClassifier with logistic loss (log_loss), enabling probabilistic binary classification. The model employs:

- Constant learning rate (0.01)
- Maximum 100 training epochs
- Tolerance of 1e-4 for convergence
- Sigmoid-based probability estimation

The use of SGD ensures computational efficiency and scalability, particularly important for high-dimensional n-gram feature spaces.

Results with 80:20 train-test split

The 80:20 split was used to evaluate robustness under a larger test set. This configuration provides a stricter validation of model generalization. The 80% of the data were used to training the models while selecting randomly from the entire dataset and the rest 20% of the data were used to measure the model performance. all the models were train and tested with the prepared dataset to ensure the fair comparison.

Performance on 3-gram features With 3-gram features, the Trigonometric FLANN achieved an accuracy of 97.37% and an AUC of 0.9909. The AUC exceeding 0.99 indicates near-perfect separability between malicious

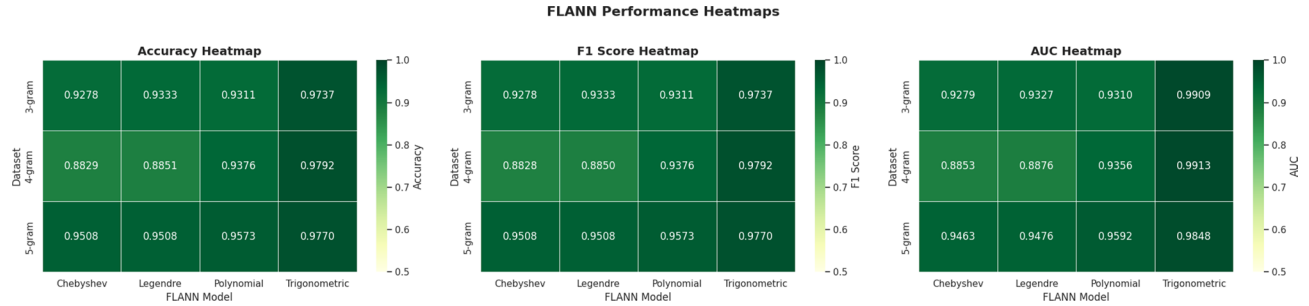


Fig. 6 Heatmap presentation of model performance comparison with respect to n-Gram feature extraction with 80:20 split.

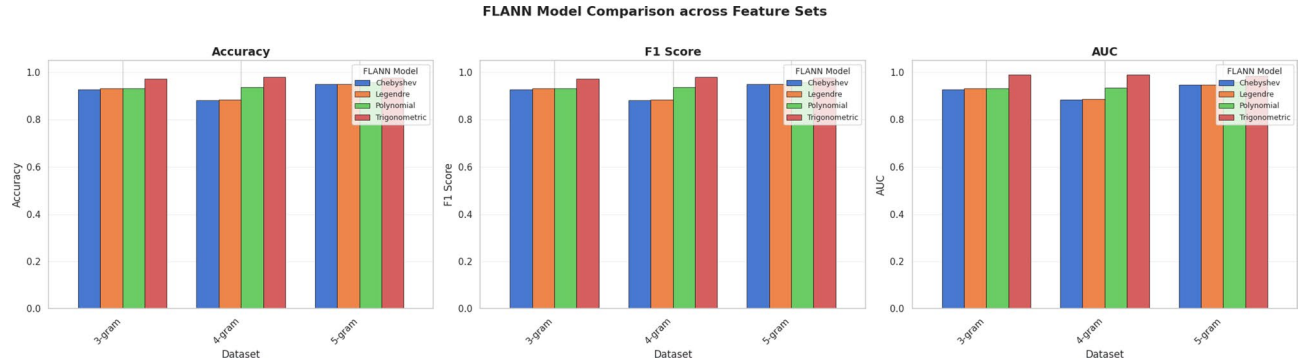


Fig. 7 Bar chart presentation of model performance comparison with respect to n-Gram feature extraction with 80:20 split.

and benign classes. Other FLANN variants continued to show moderate performance, reinforcing the superiority of the trigonometric expansion.

Performance on 4-gram features

The 4-gram representation produced one of the best overall results under the 80:20 split. The Trigonometric FLANN achieved 97.92% accuracy and an AUC of 0.9913, which was the highest AUC among all n-gram configurations in this split. This indicates that 4-gram features capture sufficient contextual information without introducing excessive sparsity. The model benefits from richer lexical sequences while maintaining generalization stability.

Performance on 5-gram features

For 5-gram features, the Trigonometric FLANN achieved an accuracy of 97.70% and an AUC of 0.9848. Although still high, the slight reduction compared to 4-gram results suggests diminishing returns with increasing n-gram order.

Higher-order n-grams increase feature dimensionality and sparsity, which may limit further performance improvements. Nonetheless, the trigonometric variant remained consistently superior to the other FLANN types. The results were more clearly visualized in the bar chart, heatmap, and AUC curve presentation in Figures 6–8.

Results with 70:30 train-test split

The 70:30 split was used to evaluate robustness under a larger test set. This configuration provides a stricter validation of model generalization. The 70% of the data were used to training the models while selecting randomly

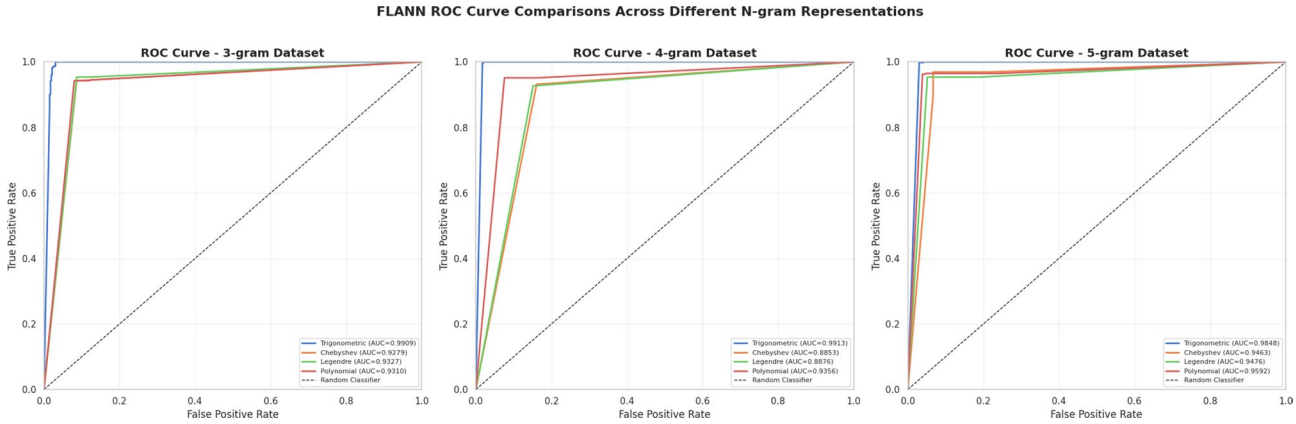


Fig. 8 AUC presentation of model performance comparison with respect to n-Gram feature extraction with 80:20 split.

from the entire dataset and the rest 30% of the data were used to measure the model performance. all the models were train and tested with the prepared dataset to ensure the fair comparison.

Performance on 3-gram features

With the 3-gram dataset, the Trigonometric FLANN model showcases outstanding results with an accuracy of 98.10% with an AUC of 0.9834. Apart from this, the other models, i.e., Polynomial FLANN, performed with an accuracy of 93.73% and an AUC of 0.9372; the Legendre FLANN accuracy was 93.65% and the AUC 0.9358; and Chebyshev FLANN had an accuracy of 93.36% and an AUC of 0.9337. The trigonometric expansion's effectiveness in capturing sequential dependencies makes up the performance gap between these models.

Performance on 4-gram features

In the 4-gram dataset, again, the Trigonometric FLANN shows its superiority with an accuracy of 97.96% and an AUC of 0.9910. Apart from that, Polynomial FLANN followed with an accuracy of 94.16% and Legendre FLANN with an accuracy of 93.07%, while Chebyshev FLANN, with an accuracy of 92.20% showed reduced effectiveness. The dimensionality of the dataset might help the traditional functional models, but the Trigonometric approach remains unaffected.

Performance on 5-gram features

For the 5-gram dataset, it outperformed all previous performance by achieving an accuracy of 98.61% and an AUC of 0.9939 with the Trigonometric FLANN model. Following this, the Polynomial FLANN and Legendre FLANN achieve an accuracy of 95.70% and 94.82%, respectively, but the Chebyshev FLANN achieves the lowest accuracy, 91.90%. Thus, the richer contextual information might help in enhancing detection; however, the Trigonometric model holds its margin of superiority among all.

The strong performance of 5-gram features under this split indicates that the trigonometric expansion effectively handles higher-dimensional representations when sufficient discriminatory patterns are present. The results were more clearly visualized in the bar chart, heatmap, and AUC curve presentation in Figures 9–11.

Validation

Across both train-test splits and all n-gram configurations, the Trigonometric FLANN consistently outperformed Chebyshev, Legendre, and Polynomial variants. This consistent dominance indicates that the selection

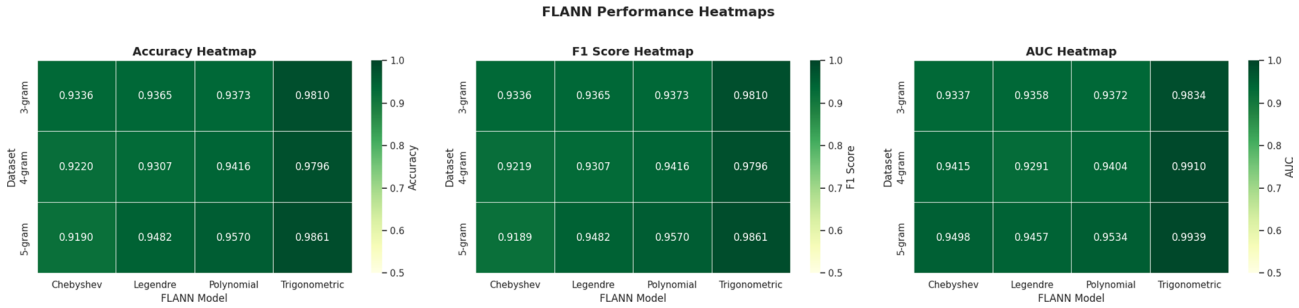


Fig. 9 Heatmap presentation of model performance comparison with respect to n-Gram Feature Extraction with 70:30 split.

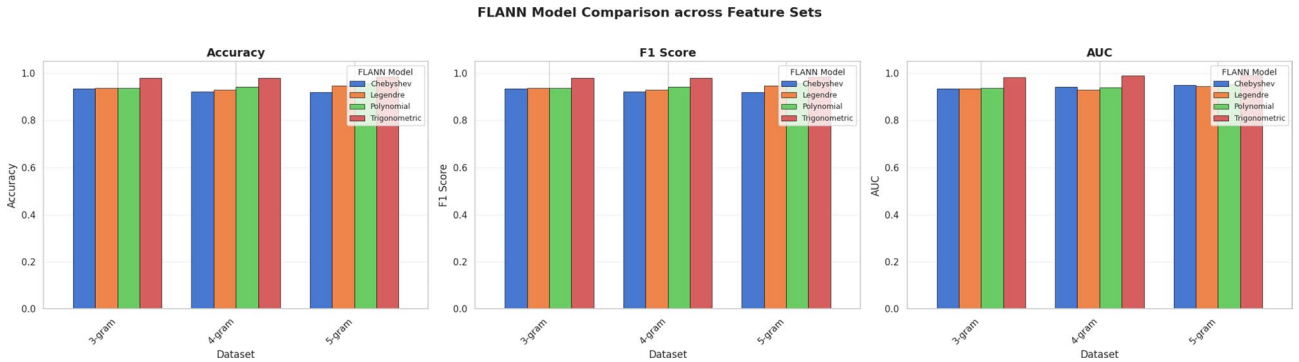


Fig. 10 Bar chart presentation of model performance comparison with respect to n-Gram feature extraction with 70:30 split.

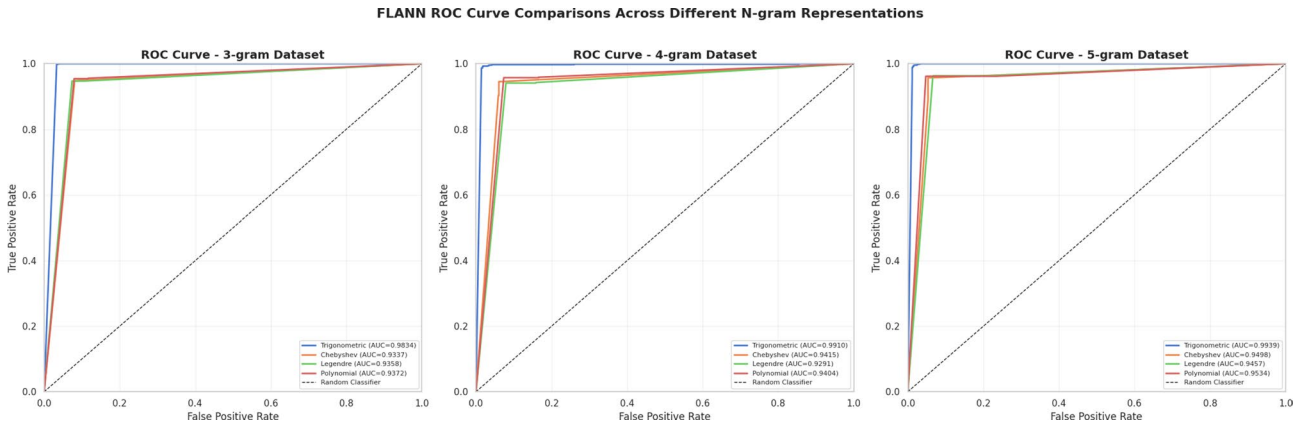


Fig. 11 AUC presentation of model performance comparison with respect to n-Gram feature extraction with 70:30 split.

of functional expansion plays a critical role in learning complex nonlinear relationships. The periodic nature of trigonometric basis functions appears to provide richer feature mapping capability compared to orthogonal polynomial expansions. As a result, the model captures subtle variations in token frequency patterns that distinguish malicious scripts from benign ones. Regarding feature representation, moderate n-gram orders (particularly 4-gram) yielded the most stable and consistently high AUC values. While increasing n-gram order enhances contextual modeling, very high orders introduce sparsity and do not guarantee proportional improvement. The consistency of model ranking across both 80:20 and 70:30 splits demonstrates robustness and low sensitivity to data partitioning. This stability indicates that the model does not heavily rely on a specific training subset and generalizes well to unseen data.

State-of-the-art and baseline comparison

A comparative analysis of the proposed model is conducted against various widely used ML models to determine whether it provides any meaningful improvement over existing approaches. Six classifiers were selected for this experiment. These classifiers were selected because they represent a diverse range of machine learning families, including linear models (SGD, Logistic Regression (LR), a Platt-calibrated Linear Support Vector Machine (SVM)), ensemble tree-based methods (Random Forest (RF), Gradient Boosting (GB)), and instance-based learning (k-Nearest Neighbors (kNN) where k=5). This enables a comprehensive comparison through different learning paradigms. The SDG classifier was trained directly on the raw (non-expanded) lexical features. It was included specifically to isolate the contribution of functional expansion. This comparison is also fair, as it shares the same optimizer, loss, and hyperparameters as the FLANN's inner classifier. These models and their parameters were selected based on prior PowerShell and script-based malware detection literature[6,14]. All models were simulated under a similar environment, i.e., a 70:30 train-test split with random seed 42, along with the same scaled features for fair evaluation. Table 6 and Figure 12 present classification metrics for every model for the three n-gram feature sets, along with the fit and prediction wall-clock time measured on the same hardware.

It was observed that Trigonometric expansion stands out among the four variants of FLANN. The same was observed in the previous results as well for the 80:20 and 70:30 splits. This baseline reproduces the similar ranking under a fully independent comparison protocol. It also presents the consistency in results of the FLANN model. On the other hand, a few baselines are competitive with, as well as slightly ahead of, Trigonometric

Table 6 FLANN variants vs. baseline classifiers, 70:30 split (`Random_State`=42).

N-gram	Model	Accuracy	F1	AUC	FPR	FNR	Fit Time (s)	Predict Time (s)
3-gram	**Trigonometric FLANN**	0.9810	0.9810	0.9834	0.0361	0.0015	11.53	2.414
3-gram	Chebyshev FLANN	0.9336	0.9336	0.9337	0.0808	0.0516	4.50	0.877
3-gram	Legendre FLANN	0.9365	0.9365	0.9358	0.0736	0.0531	4.21	0.948
3-gram	Polynomial FLANN	0.9373	0.9373	0.9372	0.0794	0.0457	5.22	2.636
3-gram	SGD-Raw	0.9891	0.9891	0.9960	0.0188	0.0029	0.98	0.018
3-gram	Logistic Regression	0.9927	0.9927	0.9979	0.0130	0.0015	0.92	0.022
3-gram	Linear SVM	1.0000	1.0000	1.0000	0.0000	0.0000	26.99	0.063
3-gram	RF	0.9942	0.9942	1.0000	0.0101	0.0015	1.97	0.151
3-gram	GB	1.0000	1.0000	1.0000	0.0000	0.0000	27.24	0.027
3-gram	k-NN	0.9643	0.9643	0.9892	0.0505	0.0206	0.01	1.897
4-gram	**Trigonometric FLANN**	0.9796	0.9796	0.9910	0.0346	0.0059	5.61	2.540
4-gram	Chebyshev FLANN	0.9220	0.9219	0.9415	0.0592	0.0973	5.12	0.864
4-gram	Legendre FLANN	0.9307	0.9307	0.9291	0.0794	0.0590	4.69	0.951
4-gram	Polynomial FLANN	0.9416	0.9416	0.9404	0.0736	0.0428	6.56	2.347
4-gram	SGD-Raw	0.9920	0.9920	0.9978	0.0115	0.0044	1.00	0.020
4-gram	LR	0.9956	0.9956	0.9985	0.0072	0.0015	0.96	0.031
4-gram	Linear SVM	1.0000	1.0000	1.0000	0.0000	0.0000	33.91	0.110
4-gram	RF	0.9949	0.9949	1.0000	0.0087	0.0015	2.76	0.148
4-gram	GB	1.0000	1.0000	1.0000	0.0000	0.0000	27.25	0.027
4-gram	k-NN	0.9737	0.9737	0.9912	0.0447	0.0074	0.01	1.979
5-gram	**Trigonometric FLANN**	0.9861	0.9861	0.9939	0.0245	0.0029	9.37	2.338
5-gram	Chebyshev FLANN	0.9190	0.9189	0.9498	0.0534	0.1091	5.14	0.844
5-gram	Legendre FLANN	0.9482	0.9482	0.9457	0.0664	0.0369	1.82	0.923
5-gram	Polynomial FLANN	0.9570	0.9570	0.9534	0.0476	0.0383	5.78	2.908
5-gram	SGD-Raw	0.9934	0.9934	0.9980	0.0087	0.0044	1.61	0.018
5-gram	LR	0.9971	0.9971	0.9986	0.0043	0.0015	0.99	0.022
5-gram	Linear SVM	1.0000	1.0000	1.0000	0.0000	0.0000	49.32	0.085
5-gram	RF	0.9978	0.9978	1.0000	0.0043	0.0000	3.57	0.185
5-gram	GB	1.0000	1.0000	1.0000	0.0000	0.0000	29.63	0.026
5-gram	k-NN	0.9825	0.9825	0.9934	0.0260	0.0088	0.01	2.014

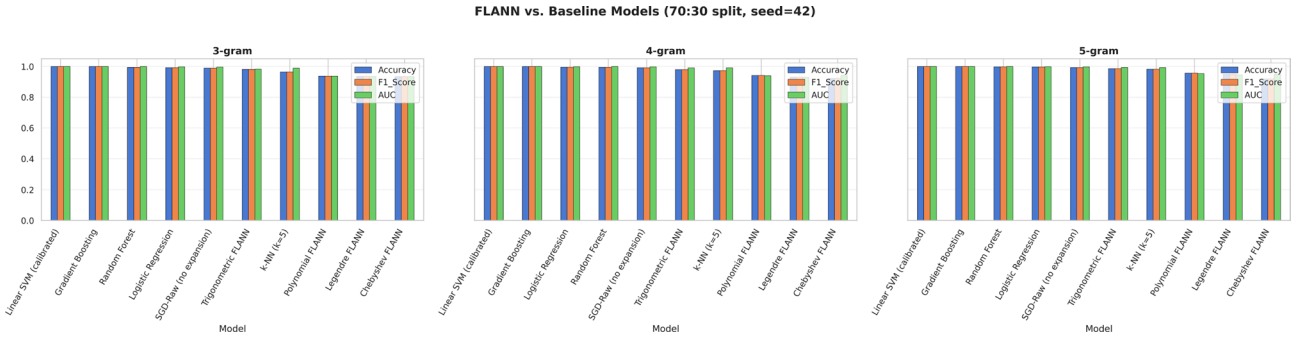


Fig. 12 Visualization of classification metrics of the Four FLANN variants against baseline classifiers, grouped by n-Gram feature set (70:30 split, `Random_State`=42).

FLANN. LR and RF exceed an accuracy of 99% on every n-gram set. In addition, Linear SVM and GB reach an accuracy of 100%, with zero false positives and zero false negatives on all three feature sets. The raw-feature SGB baseline, which uses the exact same optimizer and loss as the FLANN's internal classifier but skips the functional expansion step, also outperforms every FLANN variant. This indicates that the lexical n-gram features are close to linearly separable to begin with on this dataset. As a result, a plain linear or ensemble model can already draw a near-perfect boundary. The results of baselines should be interpreted with a caution before treating them superior to the Trigonometric FLANN. A classifier that reaches an ideal accuracy with no error is unusual for a malware detection task. These results indicate that the model fits closely to the characteristics specific to this sample set. Tree-based ensembles and instance-based methods are also prone to fitting sharp, sample-specific boundaries. This makes them sensitive to the small perturbations, which are usually introduced through obfuscation. On the other hand, a bounded, periodic expansion such as Trigonometric FLANN is comparatively less able to fit isolated outlier pattern exactly.

Ablation study: effects of N-gram order

To determine the effect of each factor, this ablation will proceed by keeping the FLANN model and the expansion order constant, while changing the number of bytes from 3 to 5 for the sequence features. A higher n-gram order results in longer, more specific byte sequences to key on but a sparser and higher-dimensional feature space. The simulation environment was kept identical for a fair evaluation. Random seed 42 was used for evaluation.

Table 7 reports the accuracy, F1, AUC, FPR, and FNR for every model at each n-gram order for the 70:30 split and the 80:20 split. Figure 13 shows the performance across all the models with the effect of different n-gram orders. Figure 14 breaks this down by model, and Table 8 displayed the results in the confusion matrices so that the raw TP/FP/FN/TN counts are visible directly for the best-performing configuration (5-gram).

It can be observed that the transition from a 3-gram to a 5-gram is helpful for the approach. In the case of the Trigonometric, Legendre, and Polynomial expansions, both the accuracy and the AUC increase with the n-gram order. Similarly, the FNR was observed to decrease. This is more important to consider than accuracy, as if the malware detector fails to detect malware (false negative), the malware will go through the system and is likely to cost much more than a false alarm. Using longer n-grams makes the reason for their help obvious; a 5-byte n-gram is a more specific fingerprint of malicious code than a 3-byte n-gram. As a result, the model has less ambiguity to resolve. The one exception is Chebyshev, its accuracy drops slightly from 3-gram to 4-gram but rises at 5-gram. This is a symptom of the numerical behavior of the Chebyshev expansion. As the dimensionality of the input increases (5001 features per n-gram level), the output of standardized features that fall outside of $[-1, 1]$ can affect the expanded feature space and make training less stable.

Secondly, the rankings across all the n-gram orders with each model are stable. The clear winner each time is trigonometric expansion. The best accuracy is achieved by the Polynomial, then Legendre, and then Chebyshev

Table 7 Ablation results Across n-Gram order, 70:30 split (`Random_State`=42). bold marks the best value per n-Gram order.

N-gram	Model	Accuracy	F1	AUC	FPR	FNR
3-gram	**Trigonometric**	**0.9810**	**0.9810**	**0.9834**	**0.0361**	**0.0015**
3-gram	Chebyshev	0.9336	0.9336	0.9337	0.0808	0.0516
3-gram	Legendre	0.9365	0.9365	0.9358	0.0736	0.0531
3-gram	Polynomial	0.9373	0.9373	0.9372	0.0794	0.0457
4-gram	**Trigonometric**	**0.9796**	**0.9796**	**0.9910**	**0.0346**	**0.0059**
4-gram	Chebyshev	0.9220	0.9219	0.9415	0.0592	0.0973
4-gram	Legendre	0.9307	0.9307	0.9291	0.0794	0.0590
4-gram	Polynomial	0.9416	0.9416	0.9404	0.0736	0.0428
5-gram	**Trigonometric**	**0.9861**	**0.9861**	**0.9939**	**0.0245**	**0.0029**
5-gram	Chebyshev	0.9190	0.9189	0.9498	0.0534	0.1091
5-gram	Legendre	0.9482	0.9482	0.9457	0.0664	0.0369
5-gram	Polynomial	0.9570	0.9570	0.9534	0.0476	0.0383

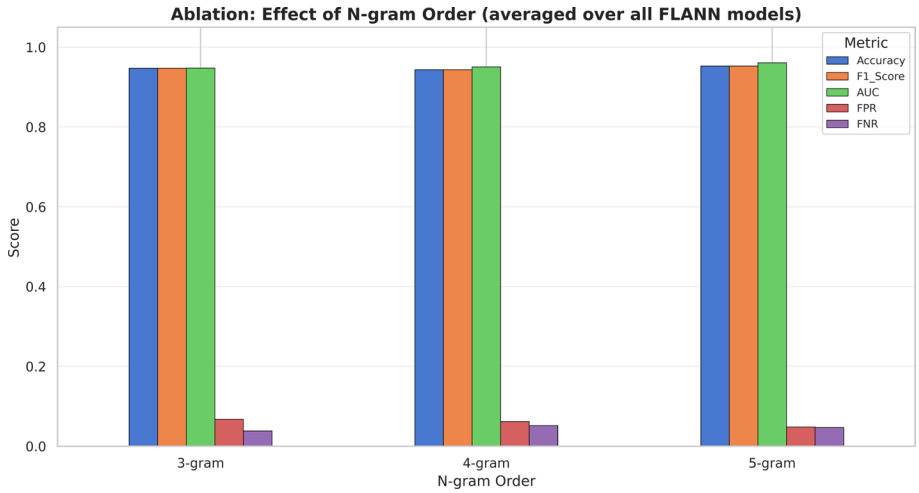


Fig. 13 Main effect of n-Gram order, averaged over all four FLANN models (70:30 split). accuracy, F1, and AUC trend upward with longer n-Grams, while FPR and FNR trend downward.

solutions (with an accuracy of 0.979–0.986 on the split 70:30). It seems natural to order these expansions in this way, since each expansion creates a bounded and periodic function of the byte count features, which can be mapped into a smooth, bounded space that a linear classifier can separate cleanly. The sensitivity to outliers, however, is not the same for all these models: polynomial and Legendre and Chebyshev terms can explode for large standardized values, which comes under notice that the the higher FPR/FNR values for these models relative to Trigonometric.

Repeatability and reproducibility analysis

An ideal model should produce reproducible and repeatable results, even if the model runs multiple times. In order to validate the repeatability of results, the proposed model is executed five times (for both splits). This test ensures that the presented results are genuine and not due to luck. It also reflects the stability in the model's performance. Various classification metrics are used to evaluate, and the results are presented in terms of mean and standard deviation for better visualization. A similar training environment and model pipeline were used for a fair evaluation. The same random seed, 42, was also used for deterministic splitting and initialization of `SGDClassifier`.

Tables 9 and 10 present the classification results of all five runs under both splits. It also presents the False Negative Rate (FNR) and False Positive Rate (FPR). Figures 15 and 16 visualizes the trends in accuracy for all

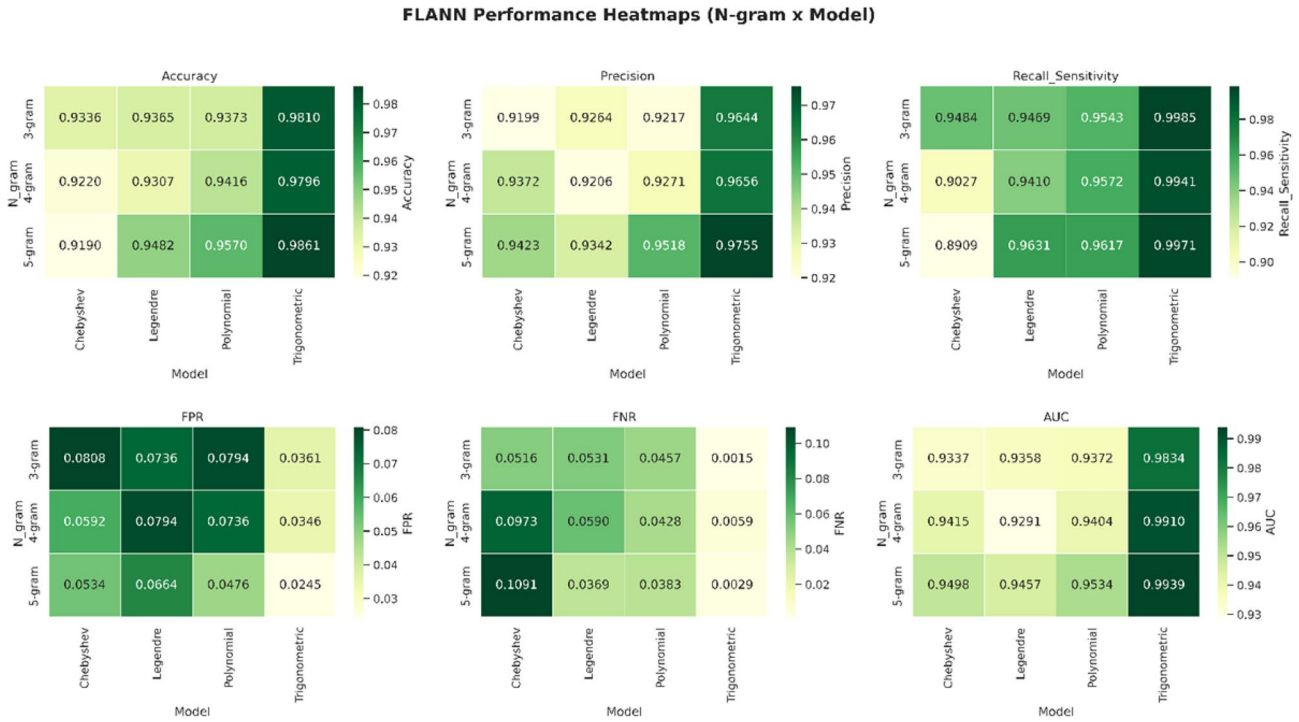


Fig. 14 Per-model heatmaps of accuracy, precision, recall, FPR, FNR, and AUC across n-Gram order.

Table 8 Confusion matrices of FLANN variants using 5-gram features.

Model	TN	FP	FN	TP
Trigonometric FLANN	676	17	2	676
Chebyshev FLANN	656	37	74	604
Legendre FLANN	647	46	25	653
Polynomial FLANN	660	33	26	652

Table 9 Reproducibility summary over 5 identical runs, 70:30 Split(`Random_State`=42). All standard deviations are 0.0, confirming the pipeline is fully deterministic.

N-gram	Model	Accuracy	Precision	Recall	F1	FPR	FNR	AUC
3-gram	Trigonometric	0.9810	0.9644	0.9985	0.9810	0.0361	0.0015	0.9834
3-gram	Chebyshev	0.9336	0.9199	0.9484	0.9336	0.0808	0.0516	0.9337
3-gram	Legendre	0.9365	0.9264	0.9469	0.9365	0.0736	0.0531	0.9358
3-gram	Polynomial	0.9373	0.9217	0.9543	0.9373	0.0794	0.0457	0.9372
4-gram	Trigonometric	0.9796	0.9656	0.9941	0.9796	0.0346	0.0059	0.9910
4-gram	Chebyshev	0.9220	0.9372	0.9027	0.9219	0.0592	0.0973	0.9415
4-gram	Legendre	0.9307	0.9206	0.9410	0.9307	0.0794	0.0590	0.9291
4-gram	Polynomial	0.9416	0.9271	0.9572	0.9416	0.0736	0.0428	0.9404
5-gram	Trigonometric	0.9861	0.9755	0.9971	0.9861	0.0245	0.0029	0.9939
5-gram	Chebyshev	0.9190	0.9423	0.8909	0.9189	0.0534	0.1091	0.9498
5-gram	Legendre	0.9482	0.9342	0.9631	0.9482	0.0664	0.0369	0.9457
5-gram	Polynomial	0.9570	0.9518	0.9617	0.9570	0.0476	0.0383	0.9534

five runs. It was observed that the standard deviation for all the tests was 0. As a result, each run's accuracy lands on precisely the same point for every model, and the five colored markers for each model fully overlap. This stage ensures that the models and engines used in the identification of malware are producing constant results, with no deviation. It also highlights that the evaluation strategy presents a similar prediction score every time, which does

Table 10 Reproducibility summary over 5 identical runs, 80:20 split (`Random_State`=42). All standard deviations are 0.0.

N-gram	Model	Accuracy	Precision	Recall	F1	FPR	FNR	AUC
3-gram	Trigonometric	0.9737	0.9777	0.9690	0.9737	0.0216	0.0310	0.9909
3-gram	Chebyshev	0.9278	0.9142	0.9425	0.9278	0.0866	0.0575	0.9279
3-gram	Legendre	0.9333	0.9151	0.9535	0.9333	0.0866	0.0465	0.9327
3-gram	Polynomial	0.9311	0.9201	0.9425	0.9311	0.0801	0.0575	0.9310
4-gram	Trigonometric	0.9792	0.9597	1.0000	0.9792	0.0411	0.0000	0.9913
4-gram	Chebyshev	0.8829	0.8499	0.9270	0.8828	0.1602	0.0730	0.8853
4-gram	Legendre	0.8851	0.8534	0.9270	0.8850	0.1558	0.0730	0.8876
4-gram	Polynomial	0.9376	0.9247	0.9513	0.9376	0.0758	0.0487	0.9356
5-gram	Trigonometric	0.9770	0.9556	1.0000	0.9770	0.0455	0.0000	0.9848
5-gram	Chebyshev	0.9508	0.9339	0.9690	0.9508	0.0671	0.0310	0.9463
5-gram	Legendre	0.9508	0.9473	0.9535	0.9508	0.0519	0.0465	0.9476
5-gram	Polynomial	0.9573	0.9519	0.9624	0.9573	0.0476	0.0376	0.9592

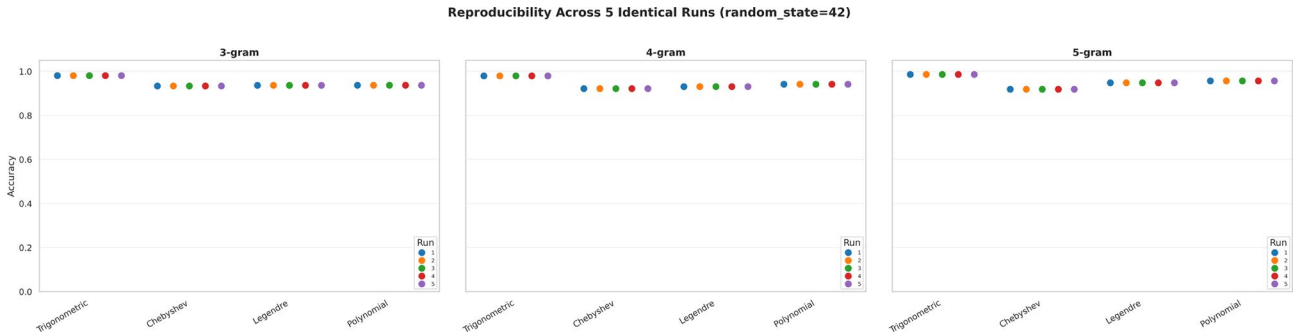


Fig. 15 Accuracy per run across 5 identical repetitions, 70:30 split. each model's 5 run-markers overlap into a single point, visually confirming zero variance.

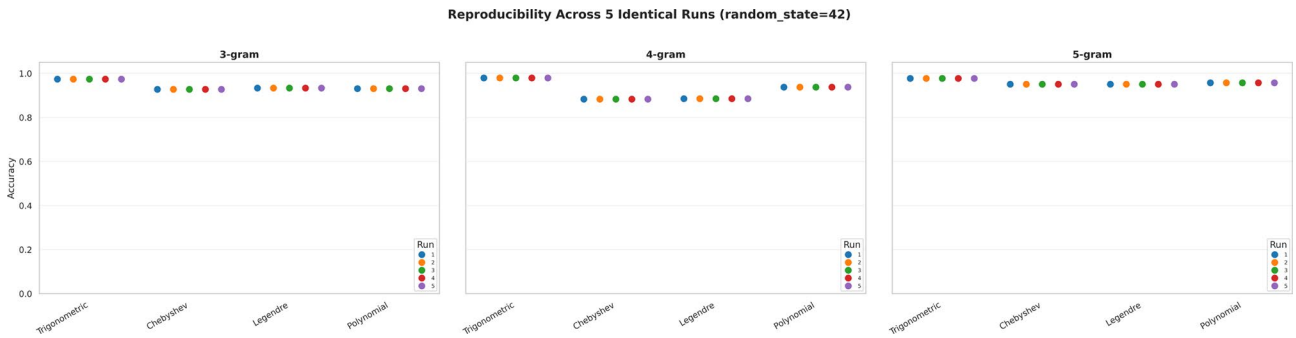


Fig. 16 Accuracy per run across 5 identical repetitions, 80:20 split.

not change the class against the threshold. These results are also a necessary outcome for a malware detection approach that does not tend to misclassify files over repeated scans.

Although the proposed model presents identical results every time it runs, it is also necessary to ensure how robust the model is against internal variations. For example, do the results change for common variations like a different initialization/split of the same dataset? Evaluation of the model utilizing different random seeds validates this analysis. It ensures that the results are not due to a favorable initialization or dataset splits. It also reflects the robustness and reproducible nature of the model. An analysis using five random seeds: 0, 21, 42, 123, and 2025, was conducted. These seeds are used randomly, which includes widely used seeds like 0 and 42 as well as arbitrary seeds like 123 and 2025. The selected seeds also present considerable diversity in both their numerical

Table 11 Seed sensitivity: Mean ± Std over five random seeds in 70:30 split.

N-gram	Model	Accuracy	F1	FPR	FNR	AUC
3-gram	Chebyshev	0.9241 ± 0.0064	0.9241 ± 0.0064	0.0873 ± 0.0067	0.0642 ± 0.0103	0.9238 ± 0.0067
3-gram	Legendre	0.9269 ± 0.0069	0.9269 ± 0.0069	0.0783 ± 0.0058	0.0678 ± 0.0103	0.9258 ± 0.0072
3-gram	Polynomial	0.9292 ± 0.0062	0.9292 ± 0.0062	0.0815 ± 0.0043	0.0597 ± 0.0124	0.9294 ± 0.0065
3-gram	Trigonometric	0.9734 ± 0.0127	0.9734 ± 0.0127	0.0317 ± 0.0153	0.0214 ± 0.0398	0.9869 ± 0.0073
4-gram	Chebyshev	0.9136 ± 0.0222	0.9135 ± 0.0222	0.0978 ± 0.0478	0.0749 ± 0.0302	0.9240 ± 0.0256
4-gram	Legendre	0.9196 ± 0.0234	0.9195 ± 0.0235	0.1043 ± 0.0434	0.0560 ± 0.0069	0.9197 ± 0.0229
4-gram	Polynomial	0.9347 ± 0.0096	0.9347 ± 0.0096	0.0732 ± 0.0096	0.0572 ± 0.0121	0.9348 ± 0.0118
4-gram	Trigonometric	0.9776 ± 0.0020	0.9776 ± 0.0020	0.0418 ± 0.0054	0.0026 ± 0.0025	0.9867 ± 0.0036
5-gram	Chebyshev	0.9442 ± 0.0181	0.9442 ± 0.0182	0.0545 ± 0.0092	0.0572 ± 0.0351	0.9494 ± 0.0066
5-gram	Legendre	0.9500 ± 0.0055	0.9500 ± 0.0055	0.0606 ± 0.0053	0.0391 ± 0.0088	0.9473 ± 0.0060
5-gram	Polynomial	0.9528 ± 0.0048	0.9528 ± 0.0048	0.0527 ± 0.0089	0.0417 ± 0.0025	0.9497 ± 0.0045
5-gram	Trigonometric	0.9829 ± 0.0032	0.9829 ± 0.0032	0.0325 ± 0.0071	0.0015 ± 0.0012	0.9906 ± 0.0032

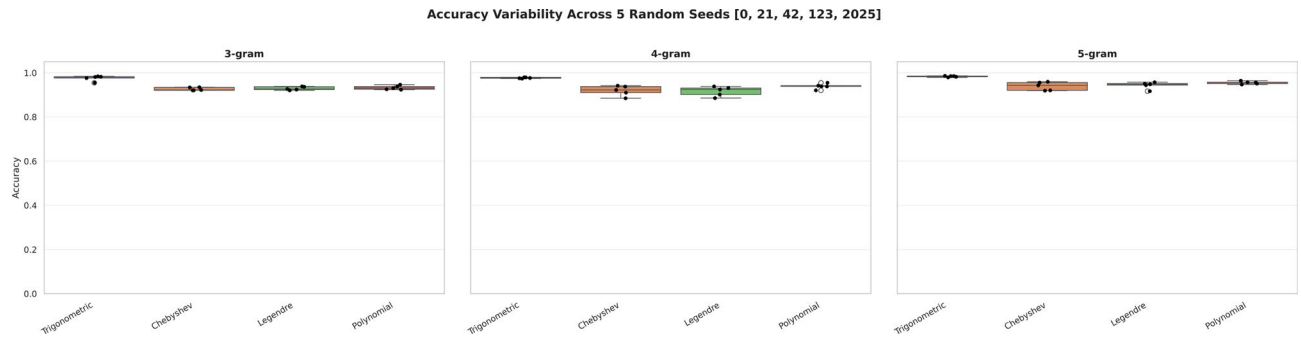


Fig. 17 Trends in accuracy across the four seeds, per model and n-Gram Order (70:30 Split). Trigonometric's boxes are tightest; chebyshev and legendre at 4-Gram show the widest spread.

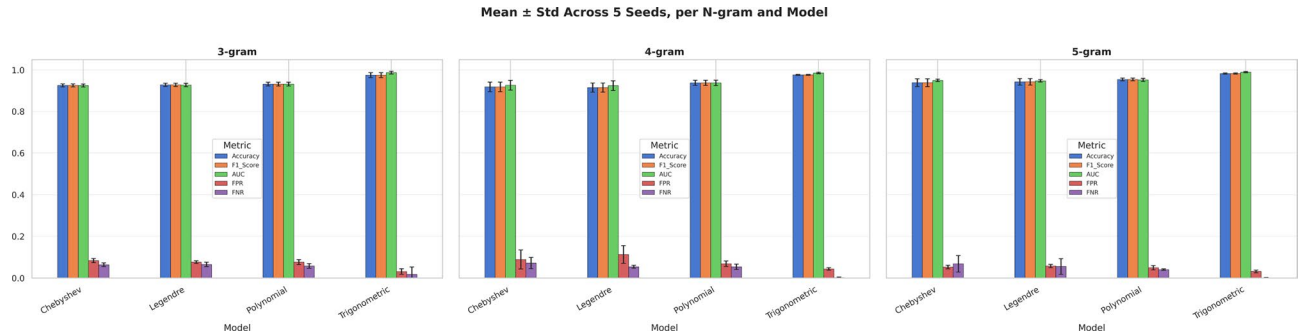


Fig. 18 Mean ± Std across seeds for accuracy, F1, AUC, FPR, and FNR, Grouped by n-Gram Order (70:30 split).

values and digit compositions, ranging from single-digit to four-digit numbers and encompassing different digit combinations. This diversity helps to ensure that the analysis is not biased toward a particular numerical pattern. This helps to conduct a comprehensive evaluation of the robustness and consistency of the results in varied initial conditions.

Table 11 reports the mean and standard deviation of accuracy, F1, FPR, FNR, and AUC over the four seeds. The accuracy variability across 5 random seeds with both the split per n-Gram and models are presented in Figures 17–19. The bottom line is reassuring; the overall accuracy difference is rather small, with an average standard deviation over all combinations of n-gram and model of 0.0101, which is around 1% only, and all accuracy

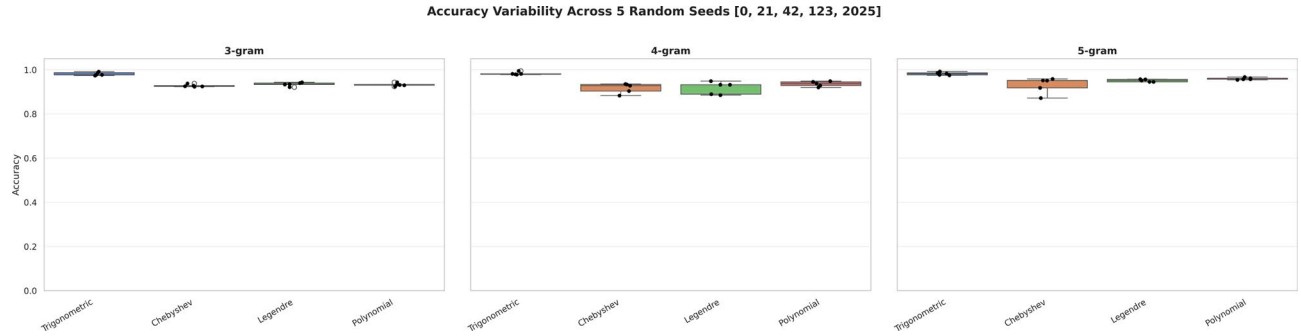


Fig. 19 Accuracy spread across seeds $\{0, 21, 42, 123, 2025\}$, 80:20 split. the same pattern holds: trigonometric is both the most accurate and the most stable model.

Table 12 Friedman Test for Equal Median Accuracy Across All Ten Models, per n-Gram Set (5 Seeds).

N-gram	Friedman statistic	p-value	Significant (α=0.05)
3-gram	44.49	1.14×10^{-6}	Yes
4-gram	44.32	1.23×10^{-6}	Yes
5-gram	43.44	1.79×10^{-6}	Yes

scores are above 0.88. Trigonometric continues to be the best model in all seeds and it stays ahead of the other three models.

Chebyshev with std $\approx$ 0.022–0.023 in accuracy, and one seed gives a visibly lower accuracy around 0.885 while others are around 0.92, and Legendre at 4-gram order has the largest instability, as does the greatest variance, which is why they are mentioned here. The variance concentrates in the cases of Chebyshev and Legendre at 4-gram order with std $\approx$ 0.022–0.023 in accuracy, and one seed gives a visibly lower accuracy around 0.885 while others are around 0.92. This is exactly what the numerical sensitivity noted in the ablation study, which states that both the Chebyshev and Legendre expansions involve polynomial recurrences that can cause values beyond a bounded range, and the train and test sets may be split in such a way that the test set has more extreme values in its features than the training set, which leads to a noticeable shift in the decision boundary. On the other hand, trigonometric remains within the range of -1 to 1 for any scale of its input, and for this reason, has the smallest variance across the seeds of all four models. These variations also present an important insight towards the prediction of the models. It ensures that the model does not spray highly confident values randomly. Instead, it analyzes the input and takes decisions accordingly.

Statistical significance testing

In order to check whether the differences in accuracy reported earlier reflect genuine differences between models, rather than the variance of a single train-test split, a statistical significance test was conducted for every FLANN model and baseline models. The simulated models were retrained and reevaluated on the same five seeds used in the seed-sensitivity analysis, i.e., 0, 21, 42, 123, 2025. This produces five paired accuracy scores per model on each n-gram set.

A Friedman test, a nonparametric, repeated-measures variant of ANOVA, is used here because the same are shared for all ten model. It was run separately for each n-gram set to test the global null hypothesis, i.e., all ten models have equal median accuracy. As presented in Table 12, the null hypothesis is rejected at $p < 0.05$ for all three n-gram sets. In addition, it was $p < 10^{-5}$ in every case. This shows that the ten models do not perform identically and that at least one genuine difference exists among them.

In order to locate where those differences lie, the Trigonometric FLANN, the best-reported FLANN variant for this investigation as per the previously conducted experiments, was compared pairwise against every other

model using the Wilcoxon signed-rank test. It is another test conducted to compare two classifiers over a small number of paired, resampled runs, with a paired *t*-test reported alongside as a parametric cross-check. The results are presented in Table 13. Both sets of *p*-values were adjusted for the nine simultaneous comparisons per n-gram set using the Holm-Bonferroni correction. With only five seeds, the Wilcoxon test's minimum attainable *p*-value is 0.0625, which after Holm correction cannot fall below the α=0.05 threshold. None of the pairwise Wilcoxon comparisons in Table 13 are significant after correction. This likely reflects the limited sample size rather than evidence that the models are equivalent. The paired *t*-test is more sensitive at this sample size and finds significant differences (Holm-adjusted $p < 0.05$) for most comparisons on the 4-gram and 5-gram sets. These include the two perfect-accuracy baselines (Linear SVM and GB), as well as LR and RF.

Table 14 summarizes the mean ± standard deviation of accuracy and F1-score across the five seeds for all ten models, extending the seed-sensitivity analysis. Figure 20 visualizes the Holm-adjusted Wilcoxon *p*-values for each n-gram set, with models to the right of the dashed reference line meeting the (unreachable, at this sample size) $p < 0.05$ mark. The paired *t*-test does detect a difference for several baselines. On the other hand, *t*-tests assume the underlying differences are approximately normal and are more easily swayed by a single favorable seed. This is a matter of concern for baselines like linear SVM and GB, whose scores show zero variance as they reach 100% on every seed. This also signifies that the model is overfitting and memorizing the dataset-specific artifacts instead of a genuine wider decision margin. On the other hand, the Trigonometric FLANN's variability for seeds was found to be std $\approx$ 0.002–0.012. This is similar to or smaller than that of the other three FLANN variants. This also aligns with the pattern already reported in Table 11. This shows that the Trigonometric FLANN model relies on underlying feature distribution instead of memorizing it. This highlights the reliability of Trigonometric FLANN over various widely used ML models.

Table 13 Pairwise comparison of trigonometric FLANN against every other model, holm-adjusted *p*-values over 5 seeds.

N-gram	Comparison (vs. TFLANN)	Mean Acc. (other)	Wilcoxon *p* (Holm)	*t*-test *p* (Holm)	Sig. (*t*-test)
3-gram	Chebyshev FLANN	0.9262	0.5625	0.0031	Yes
3-gram	Legendre FLANN	0.9291	0.5625	0.0079	Yes
3-gram	Polynomial FLANN	0.9326	0.5625	0.0125	Yes
3-gram	SGD-Raw	0.9885	0.5625	0.1358	No
3-gram	LR	0.9930	0.5625	0.0941	No
3-gram	Linear SVM	1.0000	0.5625	0.0608	No
3-gram	RF	0.9931	0.5625	0.0941	No
3-gram	GB	1.0000	0.5625	0.0608	No
3-gram	k-NN	0.9723	0.6250	0.6419	No
4-gram	Chebyshev FLANN	0.9192	0.5625	0.0121	Yes
4-gram	Legendre FLANN	0.9160	0.5625	0.0119	Yes
4-gram	Polynomial FLANN	0.9387	0.5625	0.0121	Yes
4-gram	SGD-Raw	0.9908	0.5625	0.0121	Yes
4-gram	LR	0.9959	0.5625	0.0008	Yes
4-gram	Linear SVM	1.0000	0.5625	0.0003	Yes
4-gram	RF	0.9936	0.5625	0.0027	Yes
4-gram	GB	1.0000	0.5625	0.0003	Yes
4-gram	k-NN	0.9755	0.5625	0.5691	No
5-gram	Chebyshev FLANN	0.9395	0.5625	0.0212	Yes
5-gram	Legendre FLANN	0.9434	0.5625	0.0212	Yes
5-gram	Polynomial FLANN	0.9549	0.5625	0.0011	Yes
5-gram	SGD-Raw	0.9950	0.5625	0.0212	Yes
5-gram	LR	0.9977	0.5625	0.0028	Yes
5-gram	Linear SVM	1.0000	0.5625	0.0016	Yes
5-gram	RF	0.9958	0.5625	0.0016	Yes
5-gram	GB	1.0000	0.5625	0.0016	Yes
5-gram	k-NN	0.9821	0.5625	0.2560	No

Table 14 Mean ± Std. of accuracy and F1-Score Over 5 seeds, all models, per n-Gram set.

N-gram	Model	Accuracy	F1-score
3-gram	Trigonometric FLANN	0.9755 ± 0.0119	0.9755 ± 0.0120
3-gram	Chebyshev FLANN	0.9262 ± 0.0072	0.9262 ± 0.0072
3-gram	Legendre FLANN	0.9291 ± 0.0078	0.9291 ± 0.0078
3-gram	Polynomial FLANN	0.9326 ± 0.0092	0.9326 ± 0.0092
3-gram	SGD-Raw	0.9885 ± 0.0032	0.9885 ± 0.0032
3-gram	LR	0.9930 ± 0.0021	0.9930 ± 0.0021
3-gram	Linear SVM	1.0000 ± 0.0000	1.0000 ± 0.0000
3-gram	RF	0.9931 ± 0.0021	0.9931 ± 0.0021
3-gram	GB	1.0000 ± 0.0000	1.0000 ± 0.0000
3-gram	k-NN	0.9723 ± 0.0050	0.9723 ± 0.0050
4-gram	Trigonometric FLANN	0.9768 ± 0.0024	0.9768 ± 0.0024
4-gram	Chebyshev FLANN	0.9192 ± 0.0229	0.9191 ± 0.0230
4-gram	Legendre FLANN	0.9160 ± 0.0218	0.9159 ± 0.0219
4-gram	Polynomial FLANN	0.9387 ± 0.0122	0.9387 ± 0.0122
4-gram	SGD-Raw	0.9908 ± 0.0039	0.9908 ± 0.0039
4-gram	LR	0.9959 ± 0.0012	0.9959 ± 0.0012
4-gram	Linear SVM	1.0000 ± 0.0000	1.0000 ± 0.0000
4-gram	RF	0.9936 ± 0.0020	0.9936 ± 0.0020
4-gram	GB	1.0000 ± 0.0000	1.0000 ± 0.0000
4-gram	k-NN	0.9755 ± 0.0028	0.9755 ± 0.0028
5-gram	Trigonometric FLANN	0.9832 ± 0.0029	0.9832 ± 0.0029
5-gram	Chebyshev FLANN	0.9395 ± 0.0189	0.9394 ± 0.0190
5-gram	Legendre FLANN	0.9434 ± 0.0156	0.9434 ± 0.0157
5-gram	Polynomial FLANN	0.9549 ± 0.0063	0.9549 ± 0.0063
5-gram	SGD-Raw	0.9950 ± 0.0027	0.9950 ± 0.0027
5-gram	LR	0.9977 ± 0.0010	0.9977 ± 0.0010
5-gram	Linear SVM	1.0000 ± 0.0000	1.0000 ± 0.0000
5-gram	RF	0.9958 ± 0.0019	0.9958 ± 0.0019
5-gram	GB	1.0000 ± 0.0000	1.0000 ± 0.0000
5-gram	k-NN	0.9821 ± 0.0031	0.9821 ± 0.0031

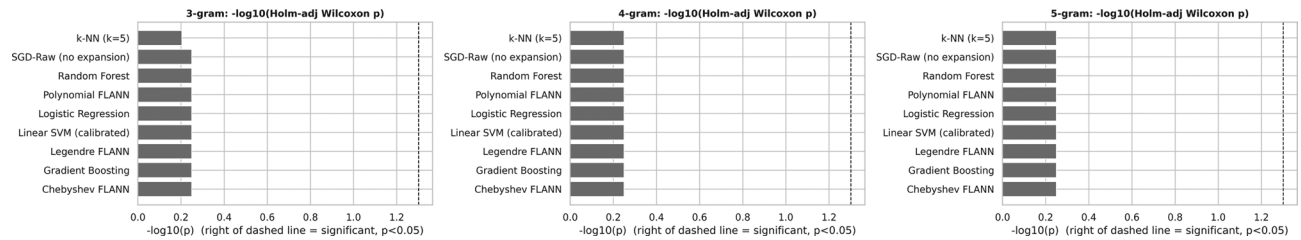


Fig. 20 Holm-adjusted wilcoxon p-values ($-\log_{10}$ Scale) for trigonometric FLANN vs. Every other model, per n-Gram Set. The dashed line marks p=0.05.

Parameter sensitivity analysis

The best-performing expansion model, the Trigonometric FLANN model, was chosen as the reference model for tuning by considering the Section 5. Considering the 4-gram dataset with a 70:30 split and `random_state`=42 a one-factor-at-a-time sweep was performed by varying the expansion order with maximum training epoch and SGD learning rate (η_0) whiling keeping the order =3, η_0 = 0.01, max_iter = 100 as per the main experiments. Table 15 displays the accuracy, F1, AUC, FPR, FNR, fit time, and expanded feature dimensionality for each setting, and Figure 21 plots the accuracy and AUC with the fit time for the three parameters.

Table 15 Hyperparameter sensitivity of the trigonometric FLANN, 4-Gram features, 70:30 split (`Random_State`=42).

Parameter	Value	Accuracy	F1	AUC	FPR	FNR	Fit Time (s)
Order	1	0.9818	0.9818	0.9972	0.0361	0.0000	3.45
Order	2	0.9905	0.9905	0.9971	0.0130	0.0059	9.93
Order	**3 (default)**	0.9796	0.9796	0.9910	0.0346	0.0059	6.66
Order	4	0.9825	0.9825	0.9926	0.0317	0.0029	8.33
Order	5	0.9788	0.9788	0.9820	0.0390	0.0029	9.93
η_0	0.001	0.9803	0.9803	0.9967	0.0361	0.0029	10.86
η_0	**0.01 (default)**	0.9796	0.9796	0.9910	0.0346	0.0059	5.60
η_0	0.05	0.9788	0.9788	0.9784	0.0404	0.0015	8.53
η_0	0.1	0.9803	0.9803	0.9819	0.0361	0.0029	10.27
max_iter	25	0.9796	0.9796	0.9910	0.0346	0.0059	5.68
max_iter	50	0.9796	0.9796	0.9910	0.0346	0.0059	5.77
max_iter	**100 (default)**	0.9796	0.9796	0.9910	0.0346	0.0059	5.65
max_iter	200	0.9796	0.9796	0.9910	0.0346	0.0059	6.58

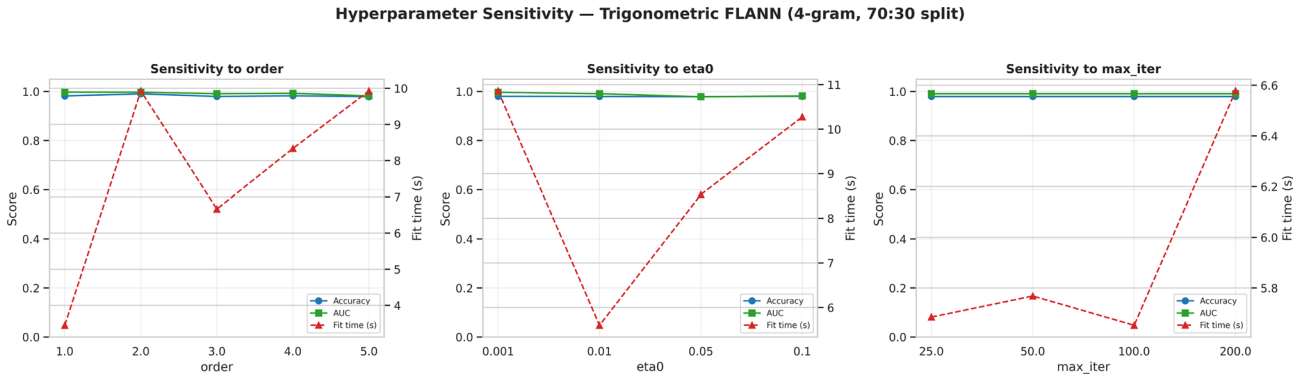


Fig. 21 Sensitivity of the Trigonometric FLANN to Expansion order, learning rate (η_0), and Training epochs (max_iter), 4-Gram features, 70:30 split. Accuracy and AUC are plotted against the left axis, fit time against the right axis.

The expansion order has the highest impact on the model. The accuracy is already high at order 1 (98.18%) and remains in a narrow band up to order 4 (98.25%); however, the AUC goes high at order 1 (0.9972) and order 2 (0.9971), and then gradually decreases to 0.9820 by order 5. The features count also increases as the expansion order increases, like in order 1, where the features are 15,003, to order 5, where the features are 55,011. So that affects the training time to increase with the number of features. Therefore, the increase in order may affect the AUC to decrease and show very little improvement in the accuracy. Also, more the training time, requires more number of feature matrix to process. Order 3, which is considered as the base throughout the experiment, provides an effective and well-balanced relationship between the feature size, training cost, and performance of the models.

In the context of learning rate, the effect on AUC is very small, but it is noticeable. Although the accuracy remains stable (97.88–98.03%) throughout all the learning rates used, $\eta_0 \in \{0.001, 0.01, 0.05, 0.1\}$. However, the AUC decreases from 0.9967 at $\eta_0 = 0.001$ to 0.9784 at $\eta_0 = 0.05$, followed by a small improvement at $\eta_0 = 0.1$. It concludes that the effect of a higher learning rate may introduce variation during the training without affecting the final accuracy significantly. Therefore, $\eta_0 = 0.01$ gives an admissible balance between training stability and performance.

It noticed that the effect of epochs has very little once the model is trained for at least 25 epochs. Accuracy, F1-score, AUC, FPR, and FNR are unchanged to four decimal places across max_iter $\in \{25, 50, 100, 200\}$. This states that in this feature set the SGD optimizer becomes stable before 100 epochs. Therefore, the max_iter=100 setting provides sufficient training and does not result in underfitting. Similar performance can be achievable with a small number of epochs.

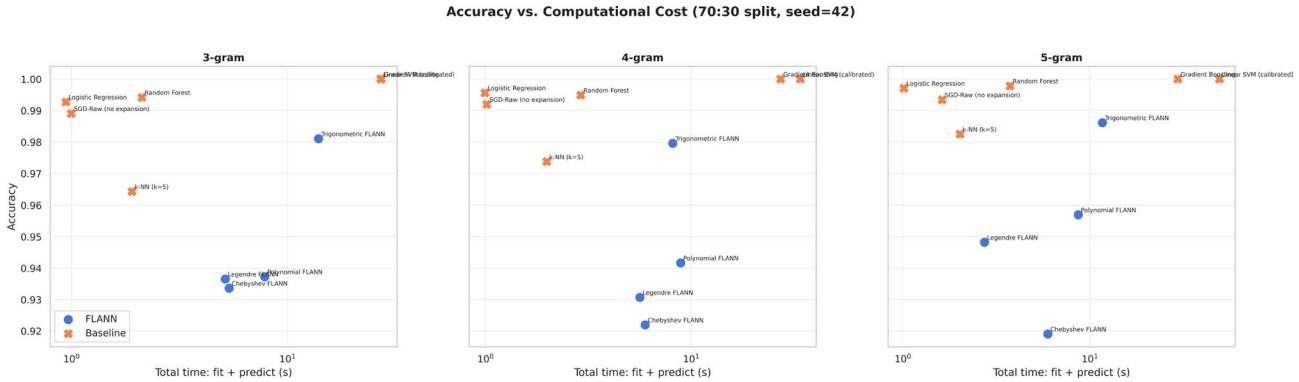


Fig. 22 Accuracy vs. total wall-clock time (Fit + Predict, Log Scale) for the Four FLANN variants and the six baseline classifiers, per n-Gram Feature Set (70:30 Split, `Random_State`=42).

Computational cost and time complexity

Beyond predictive performance, deployability depends on training and inference cost. Analytically, for n samples with d raw features and expansion order r, the functional expansion itself costs $O(n \cdot d \cdot r)$, since it is an element-wise, vectorized operation. The downstream SGD fit then costs $O(n \cdot d' \cdot \text{epochs})$, where the expanded dimensionality d' depends on the expansion type: $d' = d(2r+1)$ for the Trigonometric expansion (a sine and a cosine term per order, plus the raw feature), $d' = d(r+1)$ for Chebyshev and Legendre (which include an order-zero term), and $d' = dr$ for Polynomial. Inference then costs $O(n_{\text{test}} \cdot d')$. Among the baselines, Logistic Regression and the Linear SVM scale similarly to $O(n \cdot d \cdot \text{epochs})$ on the raw (unexpanded) features, Random Forest and Gradient Boosting scale roughly as $O(n \log n \cdot d \cdot n_{\text{estimators}})$, and k-NN has no training cost but pays $O(n_{\text{test}} \cdot n_{\text{train}} \cdot d)$ at query time, which is why it is the fastest to "fit" and the slowest to predict with in Table 6.

Empirically, this analytical picture matches the measured wall-clock times. The Trigonometric FLANN's fit time grows with the size of its expanded feature space, and its expansion also makes it markedly more expensive at inference time (2.3–2.5 s) than the linear baselines (0.02–0.11 s), since every test sample must also be passed through the sine/cosine expansion before classification. Compared against the strongest baselines, the Trigonometric FLANN trains faster than the Linear SVM and GB classifiers (which take 27–49 s to fit) but slower than LR, RF, and the raw-feature SGD baseline, all of which fit in under 3 s while matching or exceeding the FLANN's accuracy. Figure 22 plots accuracy against total (fit + predict) time on a log scale for every model and n-gram set, making this trade-off visible directly: the fastest points on the plot (SGD-Raw, LR, k-NN's near-zero fit time) already sit close to the accuracy of the FLANN variants, while the two baselines with perfect accuracy (Linear SVM, GB) sit at the most expensive end of the time axis rather than the cheapest. In other words, on this dataset, spending more computation does not automatically buy better accuracy; several inexpensive linear and ensemble baselines already sit near the top of the accuracy scale.

This cost comparison also points to a practical reason to still favor the Trigonometric FLANN over the two baselines that matched its accuracy. Both Linear SVM and GB take 27–49 seconds to fit, four to nine times longer than the Trigonometric FLANN, which undermines the low-resource, easily-retrainable design goal stated in Section 5 – a detector that needs to be periodically retrained as new PowerShell samples and obfuscation variants appear should not cost substantially more to update than the alternative. RF and LR are cheap to train, but as ensemble and margin-based linear methods respectively, neither offers the same straightforward interpretability of a fixed functional expansion, where the transformed features (sine/cosine terms of the token n-gram statistics) map back to identifiable lexical patterns in a script. On balance, the Trigonometric FLANN remains a reasonable middle ground: close to the best observed accuracy, meaningfully cheaper to train than the baselines that reach

it, and built on a bounded expansion that is less likely to be exploiting sample-specific quirks of this particular dataset.

Overall interpretation

The experimental results reported clearly demonstrate that lexical n-gram features combined with Trigonometric FLANN provide highly effective malware detection capability. The model achieves near-perfect AUC values across multiple configurations, indicating strong separability between malicious and benign PowerShell scripts. The study further confirms that basis function selection within FLANN architecture significantly influences performance. Among the tested variants, trigonometric expansion offers the most discriminative nonlinear transformation, making it the most suitab le choice for lexical-feature-based malware classification tasks. The Friedman test failed to accept the hypothesis of equality of the median accuracy for the ten models tested for each n-gram set ($p < 10^{-5}$) and the Trigonometric FLANN's accuracy was only slightly different across five different random seeds, confirming that the improvement over the other three types of expansions is a stable property of the approach and not a fluke of one particular split. The performance of some baseline classifiers performs similar or slightly higher accuracy as compared to conventional machine learning models. So the lexical n-gram features present in the dataset are nearly saturated to being linearly separable. But in consideration of less training time comparison the different FLANN models are still useful then the baseline models. Also the dependency on specific parameter is less for the model shown in the hyperparameter experiments. Even if change in learning rate, expansion order and training epochs the accuracy of the model changes very slightly within a range. This indicates the performance does not relies on any specific set of parameter to differentiate the PowerShell scripts. Noticeably, the baseline models may require more training time or may varies across different random seeds as they show similar or higher accuracy as compare to Trigonometric FLANN. This indicates that is may happens with this specific dataset as its only consist of lexical n-gram features that does not a genuinely better decision making ability. So this higher accuracy of the baseline models should not be consider advantages over the Trigonometric FLANN. Overall, the proposed approach provides a robust, high-accuracy, and computationally efficient solution for PowerShell malware detection using purely n-gram lexical features, without requiring structural or semantic representations.

Conclusion

In the process of developing an efficient PowerShell malware detection system, the reliability of the dataset played a major role. This study considered this issue by building a PowerShell dataset and introducing a systematic validation process with VirusTotal for enhancing benign and malicious labels. Along with the val idation, the multi-stage deobfuscation and normalization process extracts the hidden script from the obfuscated script before feature extraction. This pre-processing step helps to minimize common obfuscation artifacts and give a more uniform representation ready for further machine learning analysis.

The experiments showed that n-gram-based representations are efficient at capturing discriminative patterns in PowerShell scripts. Different n-gram configurations are evaluated and compared with the FLANN models in order to obtain a controlled evaluation of the proposed FLANN-based detection approach. The results show that different feature representations were more effective, suggesting that there is a need to carefully evaluate features and not just use a single one of the representations. The evidence of reliability and consistency of the proposed methods are observed by adding sensitivity and statistical analyses. In addition with these a supplementary comparison against conventional classifiers and a hyperparameter sensitivity clear that the proposed model accuracy and computational cost comparative to the baseline model.

Despite these results, this study is limited in several areas. The VirusTotal-based validation and labeling provide an effective and practical approach for large-scale sample labeling, still it should not match the expert

verification and behavioral analysis of the samples. The identification and removal of duplicate and nearly related scripts, along with the diversity of the dataset, may also affect the detection of new malware families or unseen malware. In addition, this study considered only lexical representation does not represent the semantic and behavioral features extracted from PowerShell script. The comparison with base line models also suggests that, on the present dataset, some conventional linear and ensemble classifiers give better accuracy as compare to proposed model, which suggest that the lexical features themselves are very separable for this set of malware families, and a broader and more diverse set would provide a more stringent set of the ability of the FLANN-based expansion to offer an advantages over this baseline.

Therefore, future work will focus on enhancing dataset validation by incorporating both behavioral and expert-based verification, explicitly accounting for sample diversity, and removing redundant scripts. Furthermore, to enhance the effectiveness of the feature set, hybrid representations should be considered that integrate semantic, lexical, and syntactic features, alongside ML and DL approaches, they could enhance the capability to identify advanced and evolving PowerShell threats. Adversarially altered scripts should likewise be considered to evaluate and enhance robustness against evasion attacks. Overall, the proposed framework offers a well-defined baseline for reliable PowerShell malware detection and establishes a foundation for creating a more robust, generalizable, and practically deployable detection model.

Author contributions Manish Kumar Meher: Conceptualization, methodology, data collection, implementation, and manuscript writing. Adyasha Rath: Data preprocessing, validation, feature extraction, and experimental analysis. Prabodh Kumar Sahoo: Investigation, model evaluation, and result interpretation. Aswini Kumar Samantaray: Supervision, technical review, manuscript editing, and project coordination. Ganapati Panda: Research guidance, conceptual supervision, and final manuscript review.

Data availability The data underlying this article are available in GitHub repository mkmeher/powershell-malware-dataset at[25], and can be accessed with https://doi.org/10.5281/zenodo.19634644.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Soni, A., Kumar Nanda, S., Priyadarshini, R. & Panda, G. A comprehensive review and comparative analysis of zero trust architecture: Evolution, implementation strategies, and key challenges. *Journal of Computer Security* **34**(2), 85–110. https://doi.org/10.1177/0926227X251409922 (2026).
2. Shaukat, K., Alam, T.M., Luo, S., Shabbir, S., Hameed, I.A., Li, J., Abbas, S.K.& Javed, U. A review of time-series anomaly detection techniques: A step to future perspectives. In: Future of Information and Communication Conference, pp. 865–877 (2021). https://doi.org/10.1007/978-3-030-73100-7_60 . Springer
3. Meher, M.K., Rath, A., Panda, G., Thanapati, B.B.& Puthal, D. Robust Detection of Evasive Fileless Powershell Malware: A Machine Learning Approach. In: 2025 International Conference on Artificial Intelligence and Emerging Technologies (ICAIET), pp. 1–6 (2025). https://doi.org/10.1109/ICAIET65052.2025.11211485 . IEEE
4. Singh, P., Borgohain, S.K.& Kumar, J. Performance Enhancement of SVM-based ML Malware Detection Model Using Data Preprocessing. In: 2022 2nd International Conference on Emerging Frontiers in Electrical and Electronic Technologies (ICEFEET), pp. 1–4 (2022). https://doi.org/10.1109/ICEFEET51821.2022.9848192

5. Singh, S., Lamkuche, H.S., Vishnu, D., Samara, G., Azath, H.& Qumsiyeh, E. In: Musleh Al-Sartawi, A.M.A., Nour, A.I., Abdeljawad, I. (eds.) Deep Dive into Malware Analysis and Their Behavior Patterns: A Systematic Technical Review, pp. 1931–1944. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-87584-7_140
6. Raza, M. M. et al. Malware detection and AI integration: A systematic review of current trends and future directions. *Comput. Model. Eng. Sci.* https://doi.org/10.32604/cmes.2025.074164 (2026).
7. Shaukat, K. et al. A review on security challenges in internet of things (IoT). In *2021 26th International Conference on Automation and Computing (ICAC)* 1–6 (IEEE, 2021). https://doi.org/10.23919/ICAC50006.2021.9594183.
8. Singh, P., Borgohain, S. K., Sarkar, A. K., Kumar, J. & Sharma, L. D. Feed-Forward Deep Neural Network (FFDNN)-Based Deep Features for Static Malware Detection. *International Journal of Intelligent Systems* **2023**(1), 9544481. https://doi.org/10.1155/2023/9544481 (2023).
9. Lang, J.-H. & Schreck, T. Leveraging LLMs for Memory Forensics: A Comparative Analysis of Malware Detection. *Digital Threats: Research and Practice* https://doi.org/10.1145/3748263 (2025).
10. Shaukat, K., Luo, S. & Varadharajan, V. A novel machine learning approach for detecting first-time-appeared malware. *Eng. Appl. Artif. Intell.* **131**, 107801. https://doi.org/10.1016/j.engappai.2023.107801 (2024).
11. Song, J., Kim, J., Choi, S., Kim, J. & Kim, I. Evaluations of AI-based malicious PowerShell detection with feature optimizations. *ETRI Journal* **43**(3), 549–560. https://doi.org/10.4218/etrij.2020-0215 (2021).
12. Shaukat, K., Luo, S., Chen, S. & Liu, D. Cyber threat detection using machine learning techniques: A performance evaluation perspective. In *2020 International Conference on Cyber Warfare and Security (ICCWS)* 1–6 (IEEE, 2020).
13. Salem, A., Banescu, S. & Pretschner, A. Maat: Automatically analyzing virustotal for accurate labeling and effective malware detection. *ACM Transactions on Privacy and Security (TOPS)* **24**(4), 1–35. https://doi.org/10.1145/3465361 (2021).
14. Pawlicka, A. et al. Malware Detection and Encryption: A Meta-Survey With a Unified Analytical Framework. *IEEE Access* **14**, 73006–73028. https://doi.org/10.1109/ACCESS.2026.3691124 (2026).
15. Demmese, F. A. et al. Machine learning based fileless malware traffic classification using image visualization. *Cybersecurity* **6**(1), 32. https://doi.org/10.1186/s42400-023-00170-z (2023).
16. Bozkir, A. S., Tahillioglu, E., Aydos, M. & Kara, I. Catch them alive: A malware detection approach through memory forensics, manifold learning and computer vision. *Comput. Secur.* **103**, 102166. https://doi.org/10.1016/j.cose.2020.102166 (2021).
17. Singh, N. & Tripathy, S. Unveiling the veiled: An early stage detection of fileless malware. *Comput. Secur.* **150**, 104231. https://doi.org/10.1016/j.cose.2024.104231 (2025).
18. Wu, M.-H. et al. MPSD: A Robust Defense Mechanism against Malicious PowerShell Scripts in Windows Systems. *Electronics* **13**(18), 3717. https://doi.org/10.3390/electronics13183717 (2024).
19. Shaukat, K., Luo, S. & Varadharajan, V. A novel method for improving the robustness of deep learning-based malware detectors against adversarial attacks. *Engineering Applications of Artificial Intelligence* **116**, 105461 (2022).
20. Alahmadi, A., Alkhraan, N. & BinSaeedan, W. MPSAutodetect: A malicious powershell script detection model based on stacked denoising auto-encoder. *Comput. Secur.* **116**, 102658. https://doi.org/10.1016/j.cose.2022.102658 (2022).
21. Tsai, M..-H., Lin, C..-C., He, Z..-G., Yang, W..-C. & Lei, C.-L. PowerDP: De-obfuscating and profiling malicious PowerShell commands with multi-label classifiers. *IEEE Access* **11**, 256–270. https://doi.org/10.1109/ACCESS.2022.3232505 (2022).
22. Mimura, M. & Tajiri, Y. Static detection of malicious PowerShell based on word embeddings. *Internet Things* **15**, 100404. https://doi.org/10.1016/j.iot.2021.100404 (2021).
23. Fang, Y., Zhou, X.& Huang, C. Effective method for detecting malicious PowerShell scripts based on hybrid features⋆. Neurocomputing 448, 30–39 (2021) https://doi.org/10.1016/j.neucom.2021.03.117
24. Kim, K., Shin, J., Park, J.-G. & Kim, J.-T. Performance evaluations of AI-based obfuscated and encrypted malicious script detection with feature optimization. *ETRI Journal* **47**(4), 753–770. https://doi.org/10.4218/etrij.2024-0255 (2025).
25. Xiong, C. et al. Generic, efficient, and effective deobfuscation and semantic-aware attack detection for PowerShell scripts. *Frontiers of Information Technology & Electronic Engineering* **23**(3), 361–381. https://doi.org/10.1631/FITEE.2000436 (2022).
26. Zhang, S., Li, S., Lu, J. & Yang, W. Power-ASTNN: A deobfuscation and AST neural network enabled effective detection method for malicious PowerShell scripts. *Comput. Secur.* **154**, 104441. https://doi.org/10.1016/j.cose.2025.104441 (2025).
27. Yang, X. et al. XLM4Detector: Multi-stage deobfuscation and semantic-driven Excel 4.0 macro malware detection. *IEEE Internet Things J.* https://doi.org/10.1109/JIOT.2025.3574724 (2025).
28. Shaukat, K. et al. Performance comparison and current challenges of using machine learning techniques in cybersecurity. *Energies* **13**(10), 2509 (2020).
29. Shaukat, K. et al. The impact of artificial intelligence and robotics on the future employment opportunities. *Trends in Computer Science and Information Technology* **5**(1), 050–054 (2020).
30. Shaukat, K., Luo, S., Varadharajan, V., Hameed, I. A. & Xu, M. A survey on machine learning techniques for cyber security in the last decade. *IEEE access* **8**, 222310–222354 (2020).

31. Shaukat, K., Rubab, A., Shehzadi, I. & Iqbal, R. A socio-technological analysis of cyber crime and cyber security in pakistan. *Transylvanian Review* **1**, 84 (2017).
32. Microsoft Corporation: About Scripts – PowerShell Documentation. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_scripts?view=powershell-7.5. Accessed: 2025-11-27 (2024)
33. Jena, S., Soni, A., Rout, J.& Nanda, S.K. Zero-Day Exploits in Web Applications: Detection, Response, and Prevention. In: 2024 3rd International Conference for Advancement in Technology (ICONAT), pp. 1–6 (2024). https://doi.org/10.1109/ICONAT61936.2024.10774961
34. Bao, P.T. Obfuscated PowerShell Dataset. https://www.kaggle.com/datasets/phamthaibao/obfuscated-dataset-powershell. Accessed: 2025-11-27 (2024)
35. Fleschutz: PowerShell Scripts Collection. https://github.com/fleschutz/PowerShell. Accessed: 2025-11-27 (2024)
36. abuse.ch: Malware Bazaar. https://bazaar.abuse.ch/browse/. Accessed: 2025-11-27 (2024)
37. Malandrone, G.M., Virdis, G., Giacinto, G., Maiorca, D., et al. Powerdecode: a powershell script decoder dedicated to malware analysis. In: Proceedings of the Italian Conference on Cybersecurity, ITASEC 2021, vol. 2940, pp. 219–232 (2021). CEUR-WS. org. https://ceur-ws.org/Vol-2940/paper19.pdf
38. Varlioglu, S. Detecting PowerShell-based Fileless Cryptojacking Attacks Using Machine Learning. PhD thesis, University of Cincinnati (2025). 10.48550/arXiv.2602.18285
39. Ali, A., Hwang, R..-H. & Lin, Y..-D. Evaluating Cyber Threat Intelligence: Accuracy, Completeness, Relevance, and Freshness. *IEEE Access* **13**, 157863–157876. https://doi.org/10.1109/ACCESS.2025.3606477 (2025).
40. VirusTotal: VirusTotal File Upload. https://www.virustotal.com/gui/home/upload. Accessed: 2025-11-27 (2025)
41. Kantchelian, A., Tschantz, M.C., Afroz, S., Miller, B., Shankar, V., Bachwani, R., Joseph, A.D. & Tygar, J.D. Better malware ground truth: Techniques for weighting anti-virus vendor labels. In: Proceedings of the 8th ACM Workshop on Artificial Intelligence and Security, pp. 45–56 (2015). https://doi.org/10.1145/2808769.2808780
42. Meher, M.K. Power-Shell-Dataset. https://github.com/mkmeher/powershell-malware-dataset. Version 1.0 (2025). https://doi.org/10.5281/zenodo.19634644

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Manish Kumar Meher[1] · Adyasha Rath[1] · Prabodh Kumar Sahoo[2] · Aswini Kumar Samantaray[3] · Ganapati Panda[4]

✉ Manish Kumar Meher
manishkumarmeher.official@gmail.com

✉ Prabodh Kumar Sahoo
sahooprabodhkumar@gmail.com

✉ Aswini Kumar Samantaray
aswini.samantaray@manipal.edu

[1] Computer Science and Engineering, C.V.Raman Global University, VidyaNagar, Mahura, Bhubaneswar 752054, Odisha, India

[2] Dept. of Mechatronics, Parul Institute of Technology, Parul University, Vadodara, Gujarat, India

[3] Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India

[4] ECE, C.V.Raman Global University, VidyaNagar, Mahura, Bhubaneswar 752054, Odisha, India